University of Antwerp
Faculty of Science

Computer Networks (2025-2026)
Part 2: Application Layer

Jeroen Famaey
jeroen.famaey@uantwerpen.be

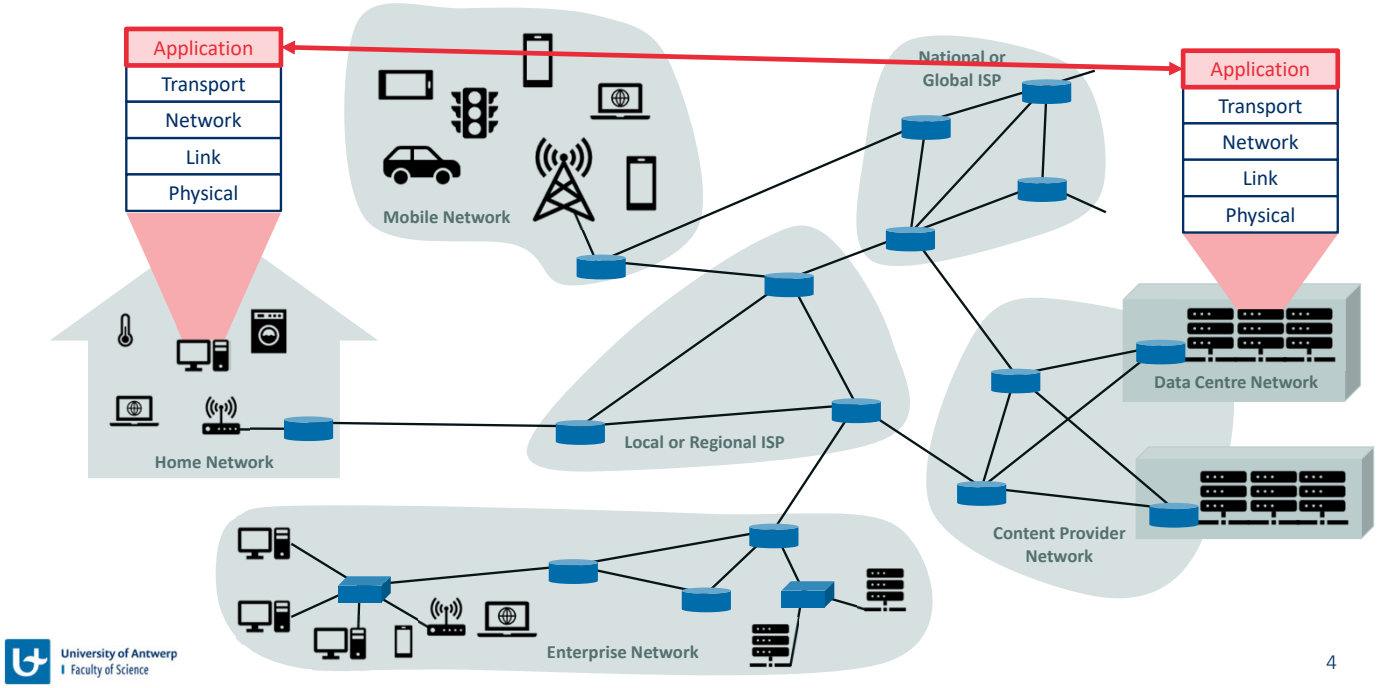
Table of contents

Network applications are the *raisons d'être* of a computer network—if we couldn't conceive of any useful applications, there wouldn't be any need for networking infrastructure and protocols to support them. Since the Internet's inception, numerous useful and entertaining applications have indeed been created. These applications have been the driving force behind the Internet's success, motivating people in homes, schools, governments, and businesses to make the Internet an integral part of their daily activities.

Internet applications include the **classic text-based applications** that became popular in the 1970s and 1980s: text e-mail, remote access to computers, file transfers, and newsgroups. They include *the* killer application of the mid-1990s, the **World Wide Web**, encompassing Web surfing, search, and electronic commerce. Since the beginning of new millennium, new and highly compelling applications continue to emerge, including **voice over IP and video conferencing** such as Skype, Facetime, and Google Hangouts; **user generated video** such as YouTube and **movies on demand** such as Netflix; and multiplayer online games such as Second Life and World of Warcraft. During this same period, we have seen the emergence of a new generation of **social networking applications**—such as Facebook, Instagram, and Twitter—which have created human networks on top of the Internet's network or routers and communication links. And most recently, along with the arrival of the smartphone and the ubiquity of 4G/5G wireless Internet access, there has been a profusion of **location based mobile apps**, including popular check-in, dating, and road-traffic forecasting apps (such as Yelp, Tinder, and Waze), **mobile payment apps** (such as WeChat and Apple Pay) and **messaging apps** (such as WeChat and WhatsApp). Clearly, there has been no slowing down of new and exciting Internet applications. Perhaps some of the readers of this text will create the next generation of killer Internet applications!

Application layer principles

Network applications run on end systems



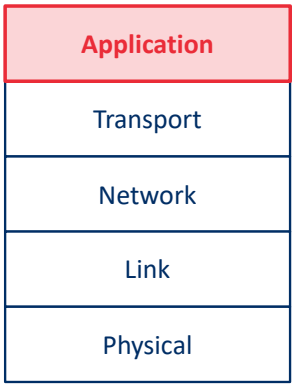
University of Antwerp
Faculty of Science

4

At the core of network application development is writing programs that run on different end systems and communicate with each other over the network. For example, in the Web application there are two distinct programs that communicate with each other: the browser program running in the user's host (desktop, laptop, tablet, smartphone, and so on); and the Web server program running in the Web server host. Servers often (but certainly not always) are housed in a data center.

Thus, when developing your new application, you need to write software that will run on multiple end systems. This software could be written, for example, in C, Java, or Python. Importantly, you do not need to write software that runs on network-core devices, such as routers or link-layer switches. Even if you wanted to write application software for these network-core devices, you wouldn't be able to do so. Network-core devices do not function at the application layer but instead function at lower layers—specifically at the network layer and below. This basic design—namely, confining application software to the end systems—has facilitated the rapid development and deployment of a vast array of network applications.

Application layer



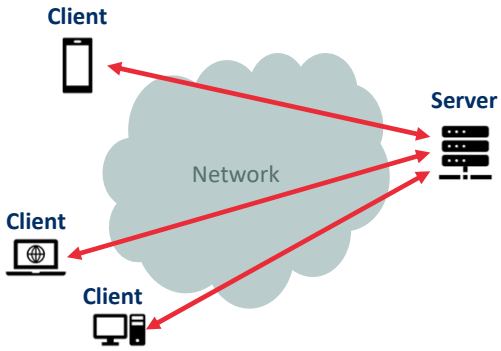
Application Layer

- Distributed over multiple end-systems
- Consists of the applications and their associated application-layer protocols
- Examples
  - HyperText Transfer Protocol (HTTP)
  - Simple Message Transfer Protocol (SMTP)
  - File Transfer Protocol (FTP)
  - Domain Name System (DNS)
  - …
- Packet of information is referred to as a **message** at this layer

The application layer is where network applications and their application-layer protocols reside. The Internet's application layer includes many protocols, such as the HTTP protocol (which provides for Web document request and transfer), SMTP (which provides for the transfer of e-mail messages), and FTP (which provides for the transfer of files between two end systems). We'll see that certain network functions, such as the translation of human-friendly names for Internet end systems like www.ietf.org to a 32-bit network address, are also done with the help of a specific application- layer protocol, namely, the domain name system (DNS). An application-layer protocol is distributed over multiple end systems, with the application in one end system using the protocol to exchange packets of information with the application in another end system. We'll refer to this packet of information at the application layer as a **message**.

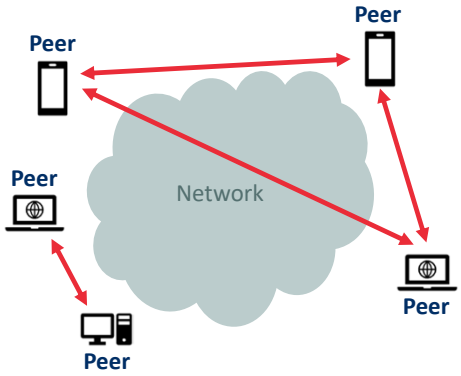
Network application architecture

Client-server architecture



- **Server** is always on (with a well-known IP address)
- **Clients** do not directly communicate with each other
- Data centres offer powerful virtual servers
- Examples: Web, FTP, Telnet, e-mail

Peer-to-peer architecture



- Minimal or no reliance on always-on servers
- Direct communication between **peers**
- Self-scaling and cost-effective
- Examples: BitTorrent, Bitcoin network, Skype (hybrid)

University of Antwerp
Faculty of Science

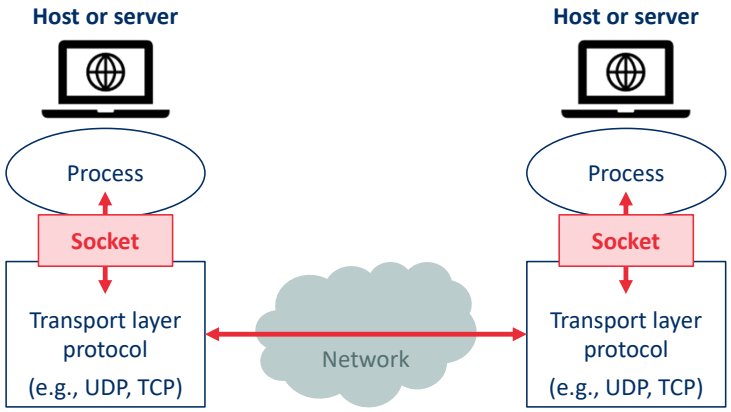
6

In a **client-server architecture**, there is an always-on host, called the ***server***, which services requests from many other hosts, called ***clients***. A classic example is the Web application for which an always-on Web server services requests from browsers running on client hosts. When a Web server receives a request for an object from a client host, it responds by sending the requested object to the client host. Note that with the client-server architecture, clients do not directly communicate with each other; for example, in the Web application, two browsers do not directly communicate. Another characteristic of the client-server architecture is that the server has a fixed, well-known address, called an IP address (which we'll discuss soon). Because the server has a fixed, well-known address, and because the server is always on, a client can always contact the server by sending a packet to the server's IP address. Some of the better-known applications with a client-server architecture include the Web, FTP, Telnet, and e-mail.

In a **peer-to-peer (P2P) architecture**, there is minimal (or no) reliance on dedicated servers in data centers. Instead the application exploits direct communication between pairs of intermittently connected hosts, called *peers*. The peers are not owned by the service provider, but are instead desktops and laptops controlled by users, with most of the peers residing in homes, universities, and offices. Because the peers communicate without passing through a dedicated server, the architecture is called peer-to-peer. An example of a popular P2P application is the file-sharing application BitTorrent. One of the most compelling features of P2P architectures is their **self-scalability**. For example, in a P2P file-sharing application, although each peer generates workload by requesting files, each peer also adds service capacity to the system by distributing files to other peers. P2P architectures are also cost effective, since they normally don't require significant server infrastructure and server bandwidth (in contrast with clients-server designs with datacenters). However, P2P applications face challenges of security, performance, and reliability due to their highly decentralized structure.

Communication between processes

- **Client**: Process that initiates the communication session between a pair of processes
- **Server**: Process that waits to be contacted to begin the session
- **Socket**: The application programming interface (API) between the application and transport layer
- Host is identified by an **IP address** and process by a **port number**



University of Antwerp
Faculty of Science

7

Before building your network application, you also need a basic understanding of how the programs, running in multiple end systems, communicate with each other. In the jargon of operating systems, it is not actually programs but **processes** that communicate. A process can be thought of as a program that is running within an end system. Processes on two different end systems communicate with each other by exchanging **messages** across the computer network. A sending process creates and sends messages into the network; a receiving process receives these messages and possibly responds by sending messages back.

A network application consists of pairs of processes that send messages to each other over a network. For example, in the Web application a client browser process exchanges messages with a Web server process. In a P2P file-sharing system, a file is transferred from a process in one peer to a process in another peer. For each pair of communicating processes, we typically label one of the two processes as the **client** and the other process as the **server**. With the Web, a browser is a client process and a Web server is a server process. With P2P file sharing, the peer that is downloading the file is labelled as the client, and the peer that is uploading the file is labelled as the server.

You may have observed that in some applications, such as in P2P file sharing, a process can be both a client and a server. Indeed, a process in a P2P file-sharing system can both upload and download files. Nevertheless, in the context of any given communication session between a pair of processes, we can still label one process as the client and the other process as the server. We define the client and server processes as follows: "*In the context of a communication session between a pair of processes, the process that initiates the communication (that is, initially contacts the other process at the beginning of the session) is labelled as the* client*. The process that waits to be contacted to begin the session is the* server*.*"

Any message sent from one process to another must go through the underlying network. A process sends messages into, and receives messages from, the network through a software interface called a **socket**. The figure illustrates socket communication between two processes that communicate over the Internet. As shown in this figure, a socket is the interface between the application layer and the transport layer within a host. It is also referred to as the **Application Programming Interface (API)** between the application and the network, since the socket is the programming interface with which network applications are built. The application developer has control of everything on the application-layer side of the socket but has little control of the transport-layer side of the socket.

In order for a process running on one host to send packets to a process running on another host, the receiving process needs to have an address. To identify the receiving process, two pieces of information need to be specified: (1) the address of the host (**IP address**) and (2) an identifier that specifies the receiving process in the destination host (**port number**).

Transport layer services

Service types a transport layer **could** offer to applications:

- **Reliable data transfer**: Ensures that data sent by one process will arrive at the receiving process
- **Throughput**: Guarantees minimum available throughput at a specified rate (in bits/second)
- **Timing**: Guarantees a maximum end-to-end delay for transmitted data (in seconds)
- **Security**: Ensures security and privacy (e.g., confidentiality, integrity, authentication) of data

The application at the sending side pushes messages through the socket. At the other side of the socket, the transport-layer protocol has the responsibility of getting the messages to the socket of the receiving process. Many networks, including the Internet, provide more than one transport-layer protocol. When you develop an application, you must choose one of the available transport-layer protocols. How do you make this choice? Most likely, you would study the services provided by the available transport-layer protocols, and then pick the protocol with the services that best match your application's needs.

Reliable data transfer
Packets can get lost within a computer network. For example, a packet can overflow a buffer in a router, or can be discarded by a host or router after having some of its bits corrupted. For many applications—such as electronic mail, file transfer, remote host access, Web document transfers, and financial applications—data loss can have devastating consequences. Thus, to support these applications, something has to be done to guarantee that the data sent by one end of the application is delivered correctly and completely to the other end of the application. If a protocol provides such a guaranteed data delivery service, it is said to provide **reliable data transfer**.

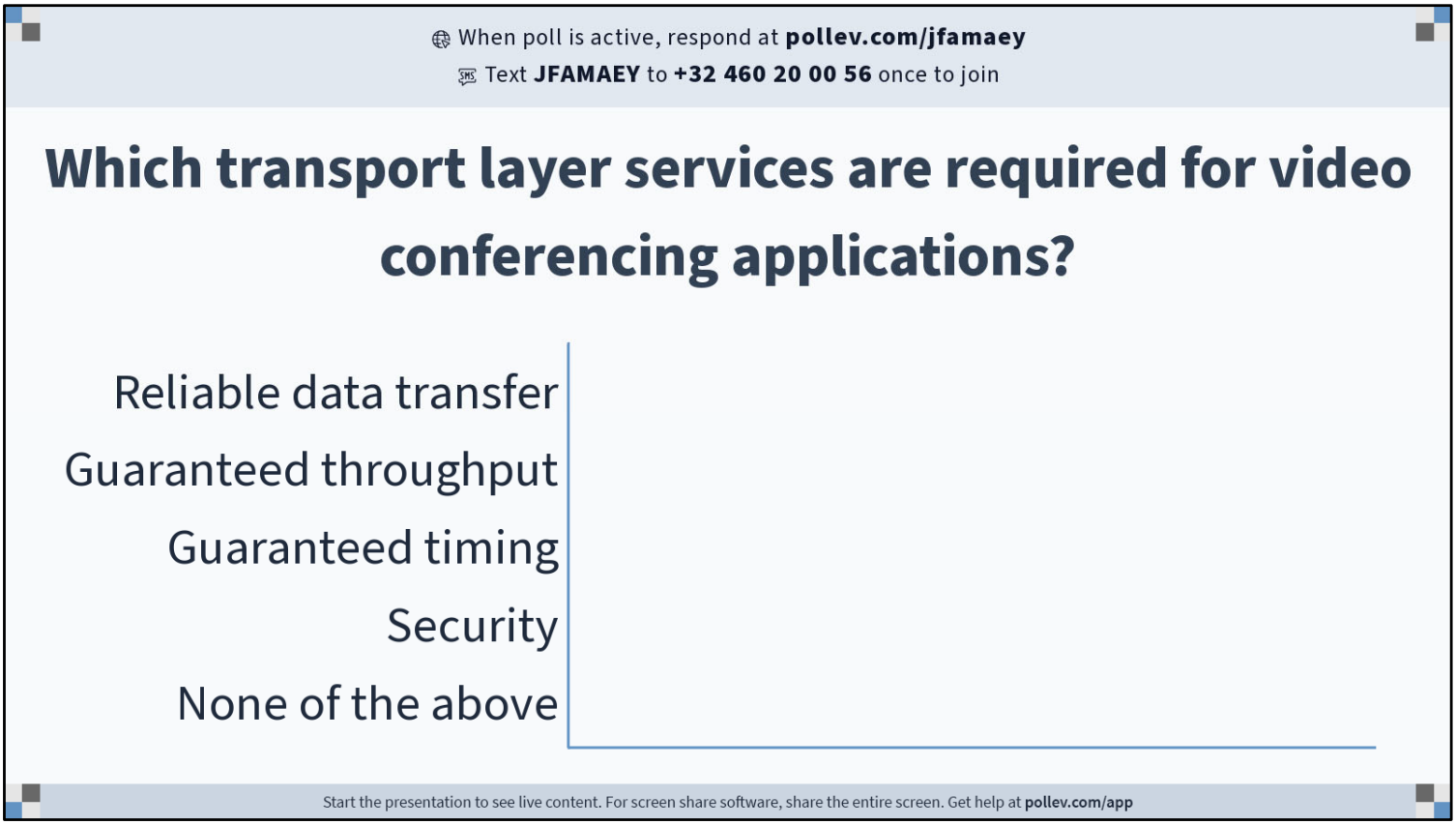
Throughput
Because other sessions will be sharing the bandwidth along the network path, and because these other sessions will be coming and going, the available throughput can fluctuate with time. These observations lead to another natural service that a transport-layer protocol could provide, namely, guaranteed available throughput at some specified rate. With such a service, the application could request a guaranteed throughput of *r* bits/sec, and the transport protocol would then ensure that the available throughput is always at least *r* bits/sec.

Timing

A transport-layer protocol can also provide timing guarantees. As with throughput guarantees, timing guarantees can come in many shapes and forms. An example guarantee might be that every bit that the sender pumps into the socket arrives at the receiver's socket no more than 100 msec later.

Security

Finally, a transport protocol can provide an application with one or more security services. For example, in the sending host, a transport protocol can encrypt all data transmitted by the sending process, and in the receiving host, the transport-layer protocol can decrypt the data before delivering the data to the receiving process. Such a service would provide confidentiality between the two processes, even if the data is somehow observed between sending and receiving processes. A transport protocol can also provide other security services in addition to confidentiality, including data integrity and end-point authentication.



Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

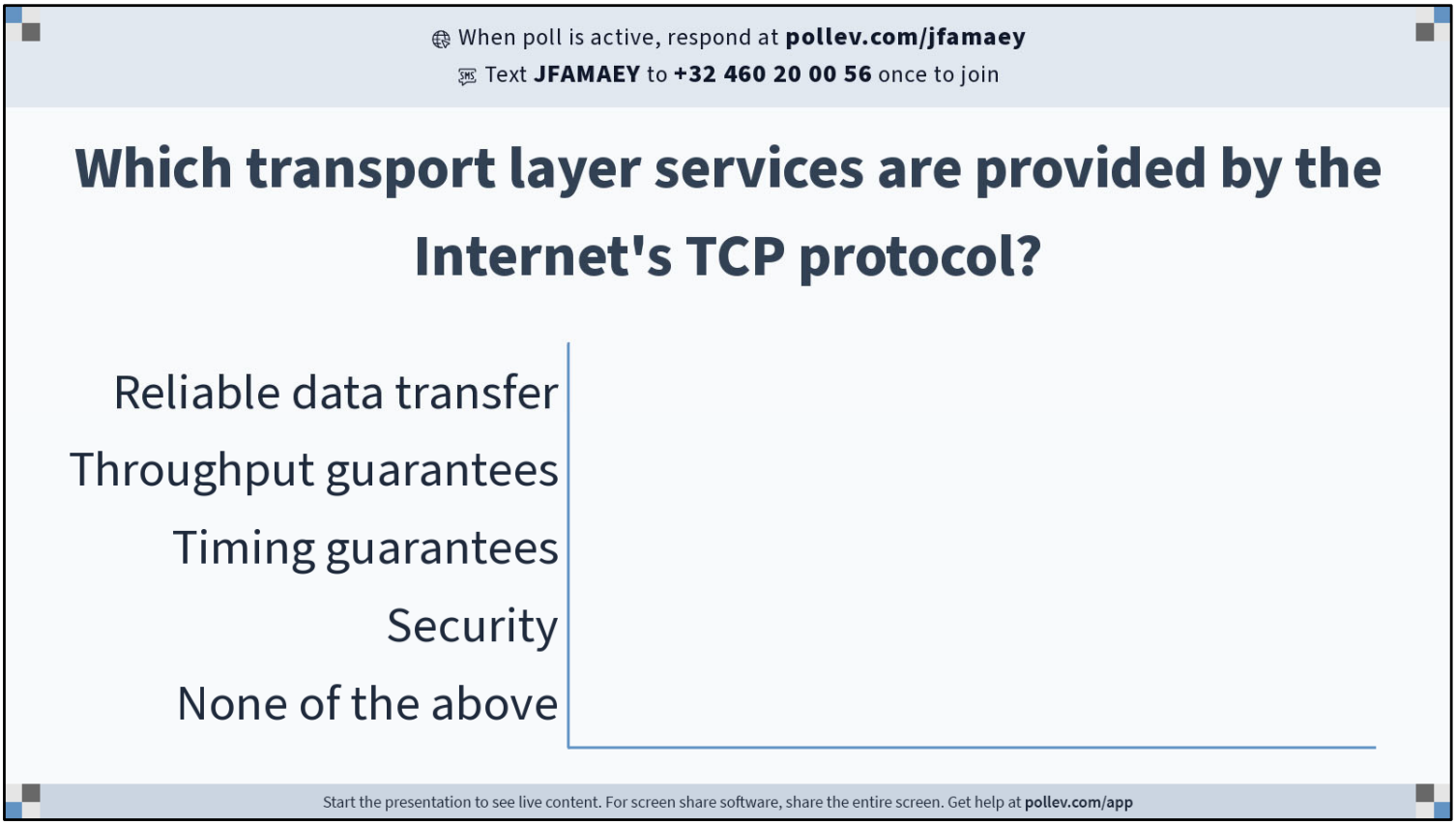
Which transport layer services are required for video conferencing applications?
https://www.polleverywhere.com/multiple_choice_polls/qR5XpCa1835HQKS1sGsQh?state=opened&flow=Default&onscreen=persist

Transport layer requirements of example applications

Application	Data Loss	Throughput	Time-Sensitive
File transfer or download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic	No
Internet telephony / video conferencing	Loss-tolerant	Audio: few kbps – 1 Mbps Video: 10 kbps – 5 Mbps	Yes: 100s of msec
Streaming stored audio / video	Loss-tolerant	Audio: few kbps – 1 Mbps Video: 10 kbps – 20 Mbps	Yes: a few seconds
Interactive games	Loss-tolerant	Few kbps – 5 Mbps	Yes: 10s – 100s of msec
Smartphone messaging	No loss	Elastic	Yes and no

When a transport-layer protocol doesn't provide reliable data transfer, some of the data sent by the sending process may never arrive at the receiving process. This may be acceptable for **loss-tolerant applications**, most notably multimedia applications such as conversational audio/video that can tolerate some amount of data loss. In these multimedia applications, lost data might result in a small glitch in the audio/video—not a crucial impairment.

Applications that have throughput requirements are said to be **bandwidth-sensitive applications**. Many current multimedia applications are bandwidth sensitive, although some multimedia applications may use adaptive coding techniques to encode digitized voice or video at a rate that matches the currently available throughput. While bandwidth-sensitive applications have specific throughput requirements, **elastic applications** can make use of as much, or as little, throughput as happens to be available. Electronic mail, file transfer, and Web transfers are all elastic applications. Of course, the more throughput, the better.



Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

Which transport layer services are provided by the Internet's TCP protocol?
https://www.polleverywhere.com/multiple_choice_polls/XDlcKICHuRAVVUoo1tfdJ?state=opened&flow=Default&onscreen=persist

Transport services provided by the Internet

Transport Layer Protocol (TCP)

- **Connection-oriented service**: Before transmitting data, a connection is set up via a handshake
- **Reliable data transfer service**: All data is guaranteed to be delivered in order and without error or loss
- **Congestion control**: Throttles sender to avoid network congestion and fairly share available bandwidth
- **Security:** Confidentiality, integrity and host authentication are offered through the TLS protocol

User Datagram Protocol (UDP)

- **Connectionless**: No handshake needed to start communication
- **Unreliable data transfer service**: Data may be lost or arrive out of order
- **No congestion control**: UDP processes can send as much data as they want/can

The Internet's transport layer **does not offer** throughput or timing guarantees!

University of Antwerp
Faculty of Science

12

The Internet (and, more generally, TCP/IP networks) makes two transport protocols available to applications, UDP and TCP. When you (as an application developer) create a new network application for the Internet, one of the first decisions you have to make is whether to use UDP or TCP. Each of these protocols offers a different set of services to the invoking applications.

TCP services

- *Connection-oriented service.* TCP has the client and server exchange transport-layer control information with each other *before* the application-level messages begin to flow. This so-called handshaking procedure alerts the client and server, allowing them to prepare for an onslaught of packets. After the handshaking phase, a **TCP connection** is said to exist between the sockets of the two processes. The connection is a full-duplex connection in that the two processes can send messages to each other over the connection at the same time. When the application finishes sending messages, it must tear down the connection.
- *Reliable data transfer service.* The communicating processes can rely on TCP to deliver all data sent without error and in the proper order. When one side of the application passes a stream of bytes into a socket, it can count on TCP to deliver the same stream of bytes to the receiving socket, with no missing or duplicate bytes.
- *Congestion-control mechanism.* The TCP congestion-control mechanism throttles a sending process (client or server) when the network is congested between sender and receiver. TCP congestion control also attempts to limit each TCP connection to its fair share of network bandwidth. This is a service for the general welfare of the Internet rather than for the direct benefit of the communicating processes.
- *Security.* Because privacy and other security issues have become critical for many applications, the Internet community has developed an enhancement for TCP, called **Transport Layer Security** (TLS). TCP-enhanced-with-TLS not only does everything that traditional TCP does but also provides critical process-to-process security services, including

encryption, data integrity, and end-point authentication. We emphasize that TLS is not a third Internet transport protocol, on the same level as TCP and UDP, but instead is an enhancement of TCP, with the enhancements being implemented in the application layer.

UDP services

UDP is a no-frills, lightweight transport protocol, providing minimal services. UDP is **connectionless**, so there is no handshaking before the two processes start to communicate. UDP provides an **unreliable data transfer service**—that is, when a process sends a message into a UDP socket, UDP provides *no* guarantee that the message will ever reach the receiving process. Furthermore, messages that do arrive at the receiving process may arrive out of order. UDP **does not include a congestion-control** mechanism, so the sending side of UDP can pump data into the layer below (the network layer) at any rate it pleases. Note, however, that the actual end-to-end throughput may be less than this rate due to the limited transmission capacity of intervening links or due to congestion.

Services not provided by the Internet

We have organized transport protocol services along four dimensions: reliable data transfer, throughput, timing, and security. Which of these services are provided by TCP and UDP? We have already noted that TCP provides reliable end-to-end data transfer. And we also know that TCP can be easily enhanced at the application layer with TLS to provide security services. But in our brief description of TCP and UDP, conspicuously missing was any mention of throughput or timing guarantees— services *not* provided by today's Internet transport protocols. Does this mean that time-sensitive applications such as Internet telephony cannot run in today's Internet? The answer is clearly no—the Internet has been hosting time-sensitive applications for many years. These applications often work fairly well because they have been designed to cope, to the greatest extent possible, with this lack of guarantee. Nevertheless, clever design has its limitations when delay is excessive, or the end-to-end throughput is limited. In summary, today's Internet can often provide satisfactory service to time-sensitive applications, but it cannot provide any timing or throughput guarantees.

Application-layer protocols

Defines type, syntax, and semantics of messages, as well as rules of message exchange

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP [RFC 5321]	TCP
Remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP 1.1 [RFC 7230], HTTP 2 [RFC 7540]	TCP
File transfer	FTP [RFC 959]	TCP
Streaming multimedia	HTTP, DASH	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], ...	UDP or TCP

May use TCP to circumvent firewalls that block UDP

The table indicates the transport protocols used by some popular Internet applications. We see that e-mail, remote terminal access, the Web, and file transfer all use TCP. These applications have chosen TCP primarily because TCP provides reliable data transfer, guaranteeing that all data will eventually get to its destination. Because Internet telephony applications (such as Skype) can often tolerate some loss but require a minimal rate to be effective, developers of Internet telephony applications usually prefer to run their applications over UDP, thereby circumventing TCP's congestion control mechanism and packet overheads. But because many firewalls are configured to block (most types of) UDP traffic, Internet telephony applications often are designed to use TCP as a backup if UDP communication fails.

We have just learned that network processes communicate with each other by sending messages into sockets. But how are these messages structured? What are the meanings of the various fields in the messages? When do the processes send the messages? These questions bring us into the realm of application-layer protocols. An **application-layer protocol** defines how an application's processes, running on different end systems, pass messages to each other. In particular, an application-layer protocol defines:

- The types of messages exchanged, for example, request messages and response messages
- The syntax of the various message types, such as the fields in the message and how the fields are delineated
- The semantics of the fields, that is, the meaning of the information in the fields
- Rules for determining when and how a process sends messages and responds to messages

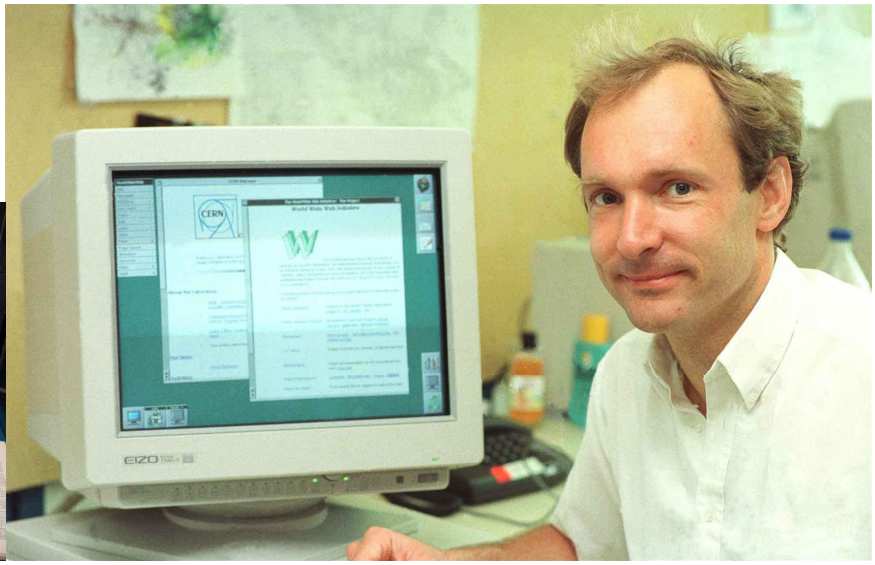
Some application-layer protocols are specified in RFCs and are therefore in the public domain. For example, the Web's application-layer protocol, HTTP 1.1 (the HyperText Transfer Protocol 1.1 [RFC 7230]), is available as an RFC. If a browser developer follows the rules of the HTTP RFC, the browser will be able to retrieve Web pages from any Web server that has also followed the

rules of the HTTP RFC. Many other application-layer protocols are proprietary and intentionally not available in the public domain. For example, Skype uses proprietary application-layer protocols.

The web and HTTP

World Wide Web (WWW)

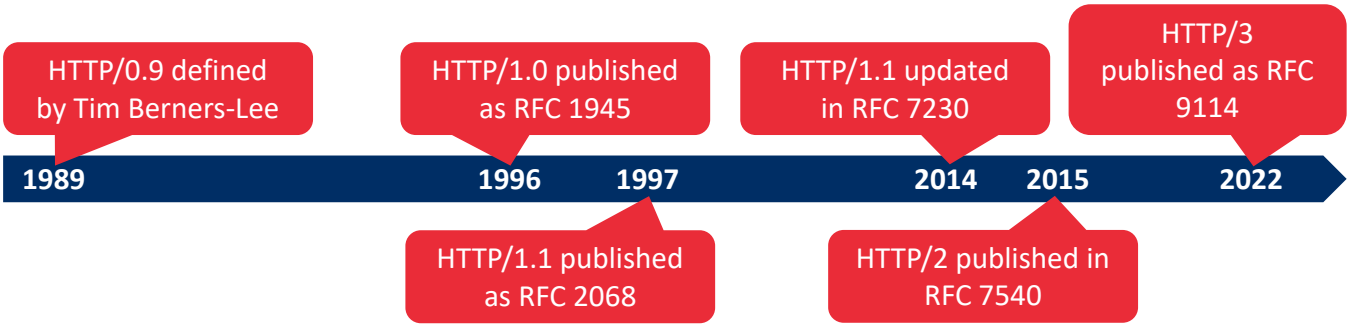
- Invented by Tim Berners-Lee between 1989 and 1991 at CERN
- Offers homogeneous information sharing using heterogeneous hardware and software



In the early 1990s, a major new application arrived on the scene—the World Wide Web. The Web was the first Internet application that caught the general public's eye. It dramatically changed how people interact inside and outside their work environments. It elevated the Internet from just one of many data networks to essentially the one and only data network. Perhaps what appeals the most to users is that the Web operates *on demand*. Users receive what they want, when they want it. This is unlike traditional broadcast radio and television, which force users to tune in when the content provider makes the content available. In addition to being available on demand, the Web has many other wonderful features that people love and cherish. It is enormously easy for any individual to make information available over the Web—everyone can become a publisher at extremely low cost. Hyperlinks and search engines help us navigate through an ocean of information. Photos and videos stimulate our senses. Forms, JavaScript, video, and many other devices enable us to interact with pages and sites. And the Web and its protocols serve as a platform for YouTube, Web-based e-mail (such as Gmail), and most mobile Internet applications, including Instagram and Google Maps.

History of HyperText Transfer Protocol (HTTP)

The HTTP protocol defines the rules of communication between web clients (browsers) and servers

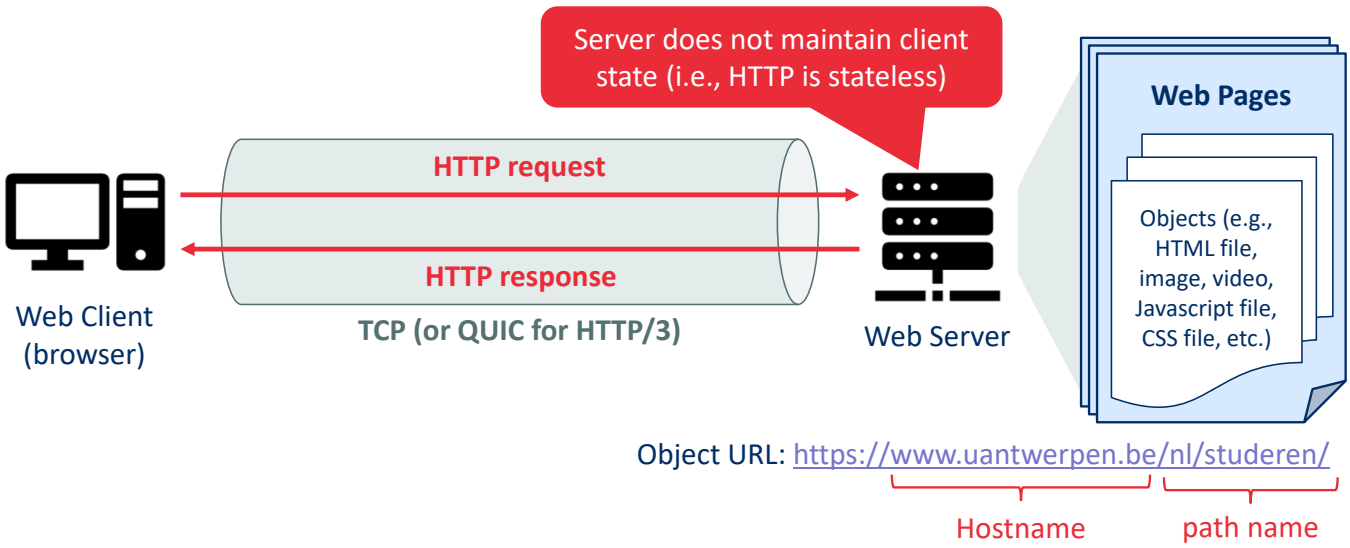


University of Antwerp
Faculty of Science

The **HyperText Transfer Protocol (HTTP)**, the Web's application-layer protocol, is at the heart of the Web. It is defined in [RFC 1945], [RFC 7230] and [RFC 7540]. HTTP is implemented in two programs: a client program and a server program. The client program and server program, executing on different end systems, talk to each other by exchanging HTTP messages. HTTP defines the structure of these messages and how the client and server exchange the messages. The original version of HTTP is called HTTP/1.0 and dates back to the early 1990's [RFC 1945]. As of 2020, the majority of HTTP transactions take place over HTTP/1.1 [RFC 7230]. However, increasingly browsers and Web servers also support a new version of HTTP called HTTP/2 [RFC 7540].

Development of HTTP was initiated by Tim Berners-Lee at CERN in 1989 and summarized in a simple document describing the behavior of a client and a server using the first HTTP protocol version that was named 0.9. That first version of HTTP protocol soon evolved into a more elaborated version that was the first draft toward a far future version 1.0. HTTP/1 was finalized and fully documented (as version 1.0) in 1996. It evolved (as version 1.1) in 1997 and then its specifications were updated in 1999 and in 2014. HTTP/2 is a more efficient expression of HTTP's semantics "on the wire", and was published in 2015; it is used by more than 45% of websites; it is now supported by almost all web browsers (96% of users). HTTP/3 is the proposed successor to HTTP/2; it is used by more than 20% of websites; it is now supported by many web browsers (73% of users). HTTP/3 uses QUIC instead of TCP for the underlying transport protocol.

HTTP at a high level



University of Antwerp | Faculty of Science

17

A **Web page** (also called a document) consists of objects. An **object** is simply a file—such as an HTML file, a JPEG image, a Javascript file, a CCS style sheet file, or a video clip—that is addressable by a single URL. Most Web pages consist of a **base HTML file** and several referenced objects. For example, if a Web page contains HTML text and five JPEG images, then the Web page has six objects: the base HTML file plus the five images. The base HTML file references the other objects in the page with the objects' URLs. Each URL has two components: the hostname of the server that houses the object and the object's path name.

Because **Web browsers** (such as Internet Explorer and Chrome) implement the client side of HTTP, in the context of the Web, we will use the words *browser* and *client* interchangeably. **Web servers**, which implement the server side of HTTP, house Web objects, each addressable by a URL. Popular Web servers include Apache and Microsoft Internet Information Server.

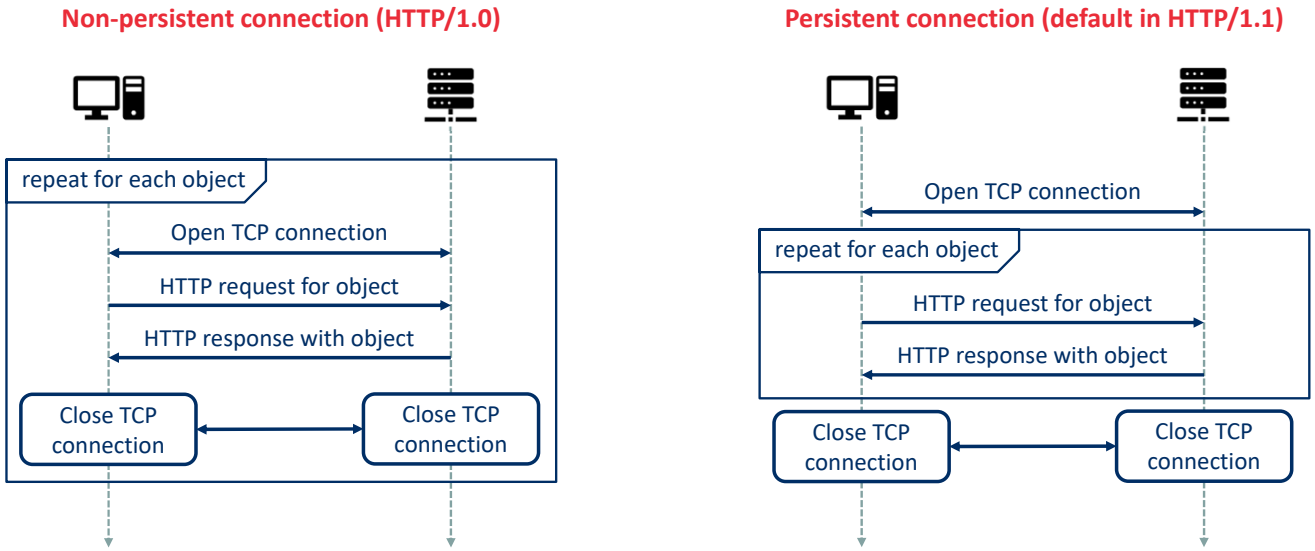
HTTP defines how Web clients request Web pages from Web servers and how servers transfer Web pages to clients. When a user requests a Web page (for example, clicks on a hyperlink), the browser sends HTTP request messages for the objects in the page to the server. The server receives the requests and responds with HTTP response messages that contain the objects.

HTTP 1.0, 1.1, and 2.0 use TCP as its underlying transport protocol (rather than running on top of UDP). The HTTP client first initiates a TCP connection with the server. Once the connection is established, the browser and the server processes access TCP through their socket interfaces. On the client side the socket interface is the door between the client process and the TCP connection; on the server side it is the door between the server process and the TCP connection. The client sends HTTP request messages into its socket interface and receives HTTP response messages from its socket interface. Similarly, the HTTP server receives request messages from its socket interface and sends response messages into its socket interface. Once the client sends a

message into its socket interface, the message is out of the client's hands and is "in the hands" of TCP. TCP provides a reliable data transfer service to HTTP. This implies that each HTTP request message sent by a client process eventually arrives intact at the server; similarly, each HTTP response message sent by the server process eventually arrives intact at the client. Here we see one of the great advantages of a layered architecture—HTTP need not worry about lost data or the details of how TCP recovers from loss or reordering of data within the network. That is the job of TCP and the protocols in the lower layers of the protocol stack.

It is important to note that the server sends requested files to clients without storing any state information about the client. If a particular client asks for the same object twice in a period of a few seconds, the server does not respond by saying that it just served the object to the client; instead, the server resends the object, as it has completely forgotten what it did earlier. Because an HTTP server maintains no information about the clients, HTTP is said to be a **stateless protocol**.

Persistent versus non-persistent connections



University of Antwerp | Faculty of Science

18

When this client-server interaction is taking place over TCP, the application developer needs to make an important decision—should each request/response pair be sent over a *separate* TCP connection, or should all of the requests and their corresponding responses be sent over the *same* TCP connection? In the former approach, the application is said to use **non-persistent connections**; and in the latter approach, **persistent connections**.

HTTP with non-persistent connections

Let's walk through the steps of transferring a Web page from server to client for the case of non-persistent connections. Let's suppose the page consists of a base HTML file and 10 JPEG images, and that all 11 of these objects reside on the same server. Further suppose the URL for the base HTML file is:

```
http://www.someSchool.edu/someDepartment/home.index
```

Here is what happens:

1. The HTTP client process initiates a TCP connection to the server www.someSchool.edu on port number 80, which is the default port number for HTTP. Associated with the TCP connection, there will be a socket at the client and a socket at the server.
2. The HTTP client sends an HTTP request message to the server via its socket. The request message includes the path name /someDepartment/home.index.
3. The HTTP server process receives the request message via its socket, retrieves the object /someDepartment/home.index from its storage (RAM or disk), encapsulates the object in an HTTP response message, and sends the response message to the client via its socket.
4. The HTTP server process tells TCP to close the TCP connection. But TCP doesn't actually terminate the connection until it knows for sure that the client has received the response message intact.

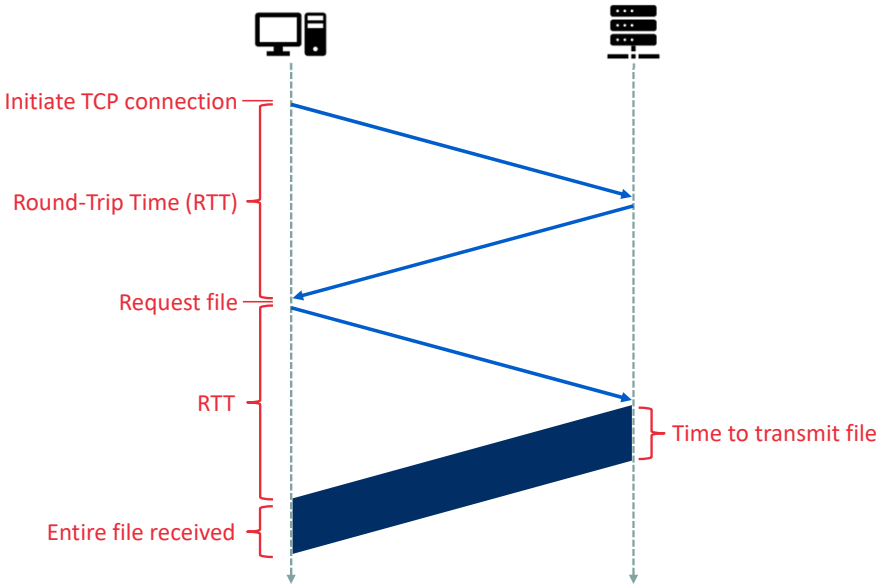
5. The HTTP client receives the response message. The TCP connection terminates. The message indicates that the encapsulated object is an HTML file. The client extracts the file from the response message, examines the HTML file, and finds references to the 10 JPEG objects.
6. The first four steps are then repeated for each of the referenced JPEG objects.

The steps above illustrate the use of non-persistent connections, where each TCP connection is closed after the server sends the object—the connection does not persist for other objects. HTTP/1.0 employs non-persistent TCP connections. Note that each non-persistent TCP connection transports exactly one request message and one response message. Thus, in this example, when a user requests the Web page, 11 TCP connections are generated.

HTTP with persistent connections

Non-persistent connections have some shortcomings. First, a brand-new connection must be established and maintained for *each requested object*. For each of these connections, TCP buffers must be allocated and TCP variables must be kept in both the client and server. This can place a significant burden on the Web server, which may be serving requests from hundreds of different clients simultaneously. With HTTP/1.1 persistent connections, the server leaves the TCP connection open after sending a response. Subsequent requests and responses between the same client and server can be sent over the same connection. In particular, an entire Web page (in the example above, the base HTML file and the 10 images) can be sent over a single persistent TCP connection. Moreover, multiple Web pages residing on the same server can be sent from the server to the same client over a single persistent TCP connection. These requests for objects can be made back-to-back, without waiting for replies to pending requests (pipelining). Typically, the HTTP server closes a connection when it isn't used for a certain time (a configurable timeout interval).

Setting up TCP connections affects delay



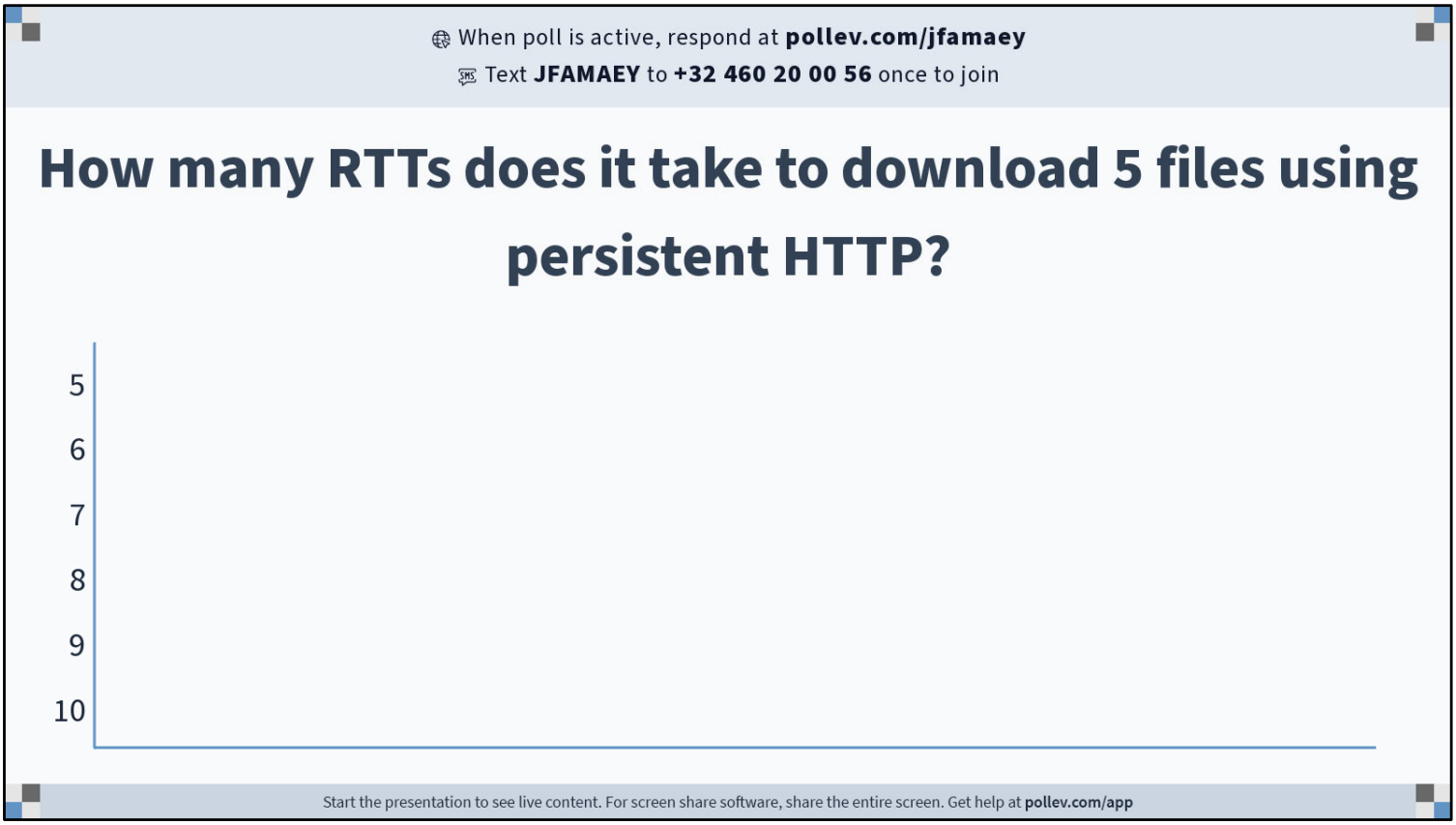
University of Antwerp
Faculty of Science

19

We define the **round-trip time (RTT)** as the time it takes for a small packet to travel from client to server and then back to the client. The RTT includes packet-propagation delays, packet-queuing delays in intermediate routers and switches, and packet-processing delays. Now consider what happens when a user clicks on a hyperlink. This causes the browser to initiate a TCP connection between the browser and the Web server; this involves a "three-way handshake"—the client sends a small TCP segment to the server, the server acknowledges and responds with a small TCP segment, and, finally, the client acknowledges back to the server. The first two parts of the three-way handshake take one RTT. After completing the first two parts of the handshake, the client sends the HTTP request message combined with the third part of the three-way handshake (the acknowledgment) into the TCP connection. Once the request message arrives at the server, the server sends the HTML file into the TCP connection. This HTTP request/response eats up another RTT. Thus, roughly, the total response time is two RTTs plus the transmission time at the server of the HTML file.

Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

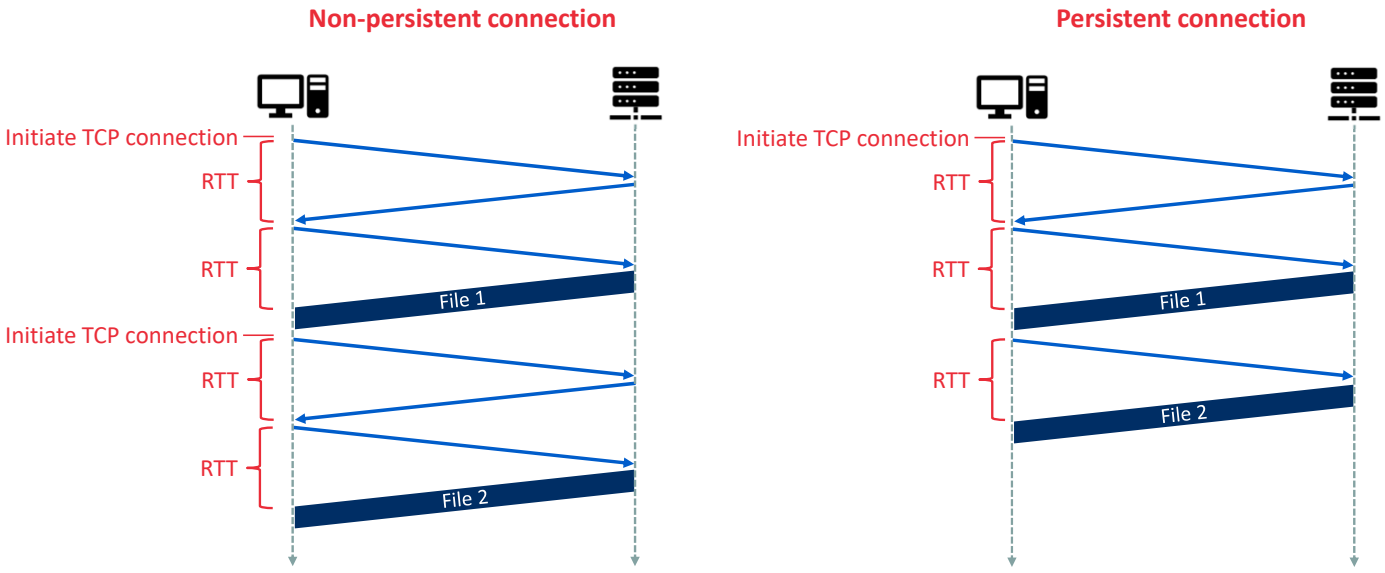
How many RTTs does it take to download 5 files using non-persistent HTTP?
https://www.polleverywhere.com/multiple_choice_polls/PXcV8TAJAbJpbMtD1Y1lS?state=opened&flow=Default&onscreen=persist



Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

How many RTTs does it take to download 5 files using persistent HTTP?
https://www.polleverywhere.com/multiple_choice_polls/NoOxNWoIFy7OtisSlIGAL?state=opened&flow=Default&onscreen=persist

Delay when requesting multiple files

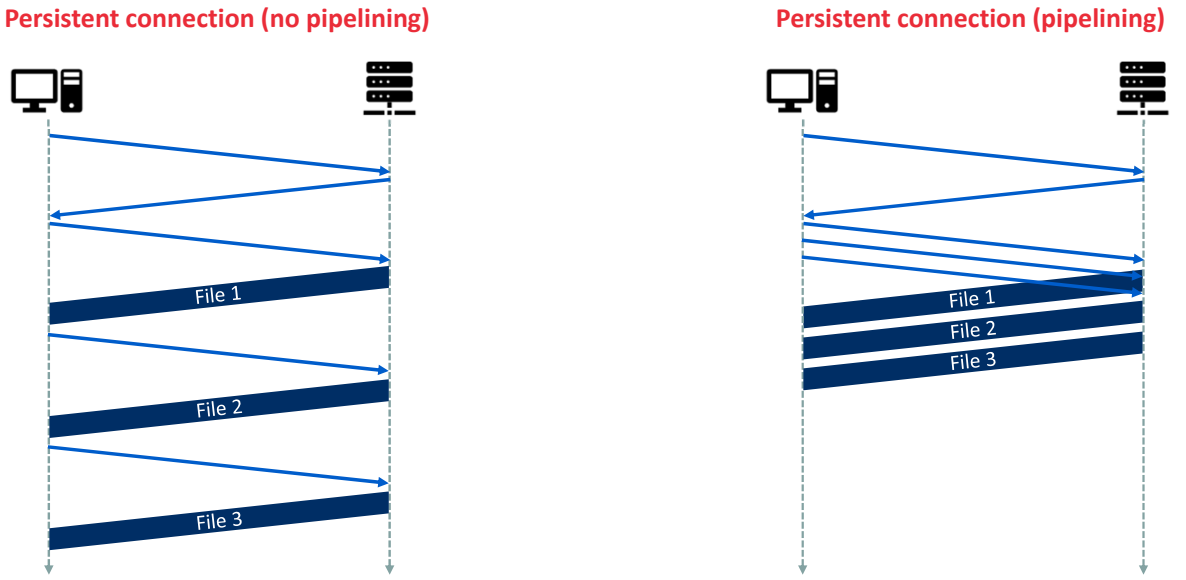


University of Antwerp
Faculty of Science

22

When using non-persistent connections, the HTTP protocol requires two complete RTTs for each file download. When using persistent connections, it requires 1 RTT to set up the initial TCP connection and then 1 RTT for each file download. To download 5 files, non-persistent HTTP thus requires 2x5=10 RTTs, while persistent HTTP only requires 1+5=6 RTTs.

HTTP/1.1 supports pipelining to further reduce RTT

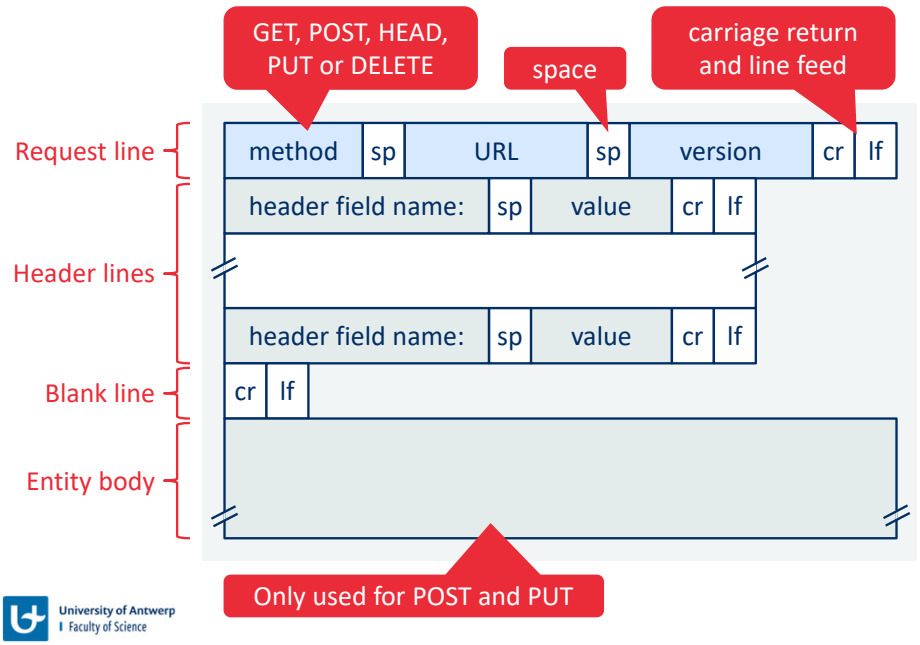


University of Antwerp
Faculty of Science

23

HTTP **pipelining** is a feature of HTTP/1.1 which allows multiple HTTP requests to be sent over a single TCP (transmission control protocol) connection without waiting for the corresponding responses. HTTP/1.1 specification requires servers to respond to pipelined requests correctly, sending back non-pipelined but valid responses even if server does not support HTTP pipelining. Despite this requirement, many legacy HTTP/1.1 servers do not support pipelining correctly, forcing most HTTP clients to not use HTTP pipelining in practice. The technique was superseded by **multiplexing via HTTP/2**, which is supported by most modern browsers.

HTTP request message format



Example:

```
GET /somedir/page.html HTTP/1.1

Host: www.uantwerpen.be
Connection: close
User-agent: Mozilla/5.0
Accept-language: nl
```

University of Antwerp | Faculty of Science

24

HTTP messages are written in ordinary ASCII text, so that your ordinary computer-literate human being can read it. The first line of an HTTP request message is called the **request line**; the subsequent lines are called the **header lines**. The request line has three fields: the method field, the URL field, and the HTTP version field. The method field can take on several different values, including GET, POST, HEAD, PUT, and DELETE. The great majority of HTTP request messages use the GET method. The GET method is used when the browser requests an object, with the requested object identified in the URL field. In the example, the browser is requesting the object /somedir/page.html. The version is self-explanatory; in this example, the browser implements version HTTP/1.1.

Now let's look at the header lines in the example. The header line Host:www.someschool.edu specifies the host on which the object resides. You might think that this header line is unnecessary, as there is already a TCP connection in place to the host. But, as we'll see in Section 2.2.5, the information provided by the host header line is required by Web proxy caches. By including the Connection:close header line, the browser is telling the server that it doesn't want to bother with persistent connections; it wants the server to close the connection after sending the requested object. The User-agent: header line specifies the user agent, that is, the browser type that is making the request to the server. Here the user agent is Mozilla/5.0, a Firefox browser. This header line is useful because the server can actually send different versions of the same object to different types of user agents. Finally, the Accept-language:header indicates that the user prefers to receive a Dutch version of the object, if such an object exists on the server; otherwise, the server should send its default version. The Accept-language: header is just one of many content negotiation headers available in HTTP. We see that the general format closely follows our earlier example. You may have noticed, however, that after the header lines (and the additional carriage return and line feed) there is an "entity body." The entity body is empty with the GET method, but is used with the POST method.

HTTP request methods

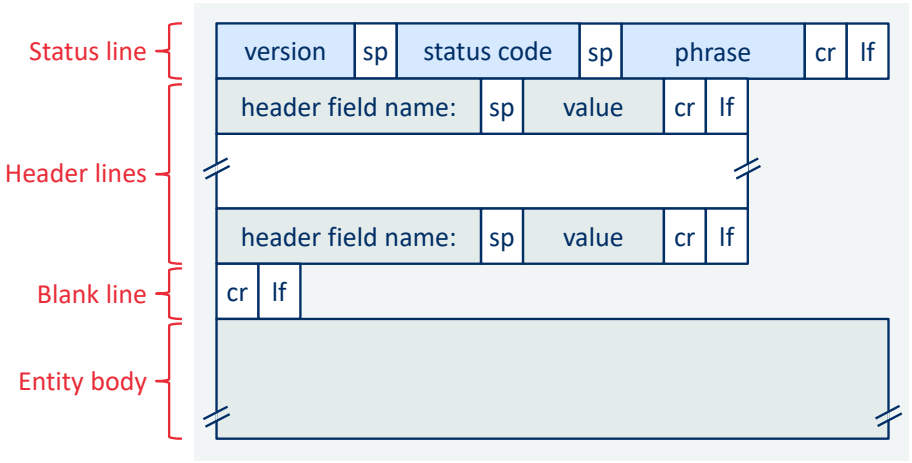
- **GET**
  - Retrieves the information identified by the request URL
  - May be used to pass form parameters via the URL (e.g., http://www.uantwerpen.be?param1=val1&param2=val2)
- **HEAD**
  - Identical to GET, except that the server will not return the message body in the response
- **POST**
  - Used to pass parameters to the server via the request body (e.g., values filled out in a form)
  - The returned information will depend on the request URL and the passed parameter values
- **PUT**
  - Allows the user to upload an object (provided in the request body) to a specific path (specified by the URL) on the web server
- **DELETE**
  - Allows the user, or an application, to delete an object (specified by the URL) on a web server
- Other methods: OPTIONS, TRACE, CONNECT

The **GET method** is used when the browser requests an object, with the requested object identified in the URL field. An HTTP client often uses the **POST method** when the user fills out a form—for example, when a user provides search words to a search engine. With a POST message, the user is still requesting a Web page from the server, but the specific contents of the Web page depend on what the user entered into the form fields. If the value of the method field is POST, then the entity body contains what the user entered into the form fields.

We would be remiss if we didn't mention that a request generated with a form does not necessarily have to use the POST method. Instead, HTML forms often use the GET method and include the inputted data (in the form fields) in the requested URL. For example, if a form uses the GET method, has two fields, and the inputs to the two fields are monkeys and bananas, then the URL will have the structure www.somesite.com/animalsearch?monkeys&bananas. In your day-to-day Web surfing, you have probably noticed extended URLs of this sort.

The **HEAD method** is similar to the GET method. When a server receives a request with the HEAD method, it responds with an HTTP message but it leaves out the requested object. Application developers often use the HEAD method for debugging. The **PUT method** is often used in conjunction with Web publishing tools. It allows a user to upload an object to a specific path (directory) on a specific Web server. The PUT method is also used by applications that need to upload objects to Web servers. The **DELETE method** allows a user, or an application, to delete an object on a Web server.

HTTP response message



Example:

```
HTTP/1.1 200 OK

Connection: close
Date: 18 Aug 2015 15:44:04
Server: Apache/2.2.3 (CentOS)
Last-Modified: 18 Aug 2015 15:11:03
Content-Length: 6821
Content-Type: text/html


data
data
data
...
```

University of Antwerp
Faculty of Science

26

Let's take a careful look at the response message example. It has three sections: an initial **status line**, six **header lines**, and then the **entity body**. The entity body is the meat of the message—it contains the requested object itself. The status line has three fields: the protocol version field, a status code, and a corresponding status message. In this example, the status line indicates that the server is using HTTP/1.1 and that everything is OK (that is, the server has found, and is sending, the requested object).

Now let's look at the header lines. The server uses the Connection: close header line to tell the client that it is going to close the TCP connection after sending the message. The Date: header line indicates the time and date when the HTTP response was created and sent by the server. Note that this is not the time when the object was created or last modified; it is the time when the server retrieves the object from its file system, inserts the object into the response message, and sends the response message. The Server: header line indicates that the message was generated by an Apache Web server; it is analogous to the User-agent: header line in the HTTP request message. The Last-Modified: header line indicates the time and date when the object was created or last modified. The Last-Modified: header, which we will soon cover in more detail, is critical for object caching, both in the local client and in network cache servers (also known as proxy servers). The Content-Length: header line indicates the number of bytes in the object being sent. The Content-Type: header line indicates that the object in the entity body is HTML text.

HTTP response status codes

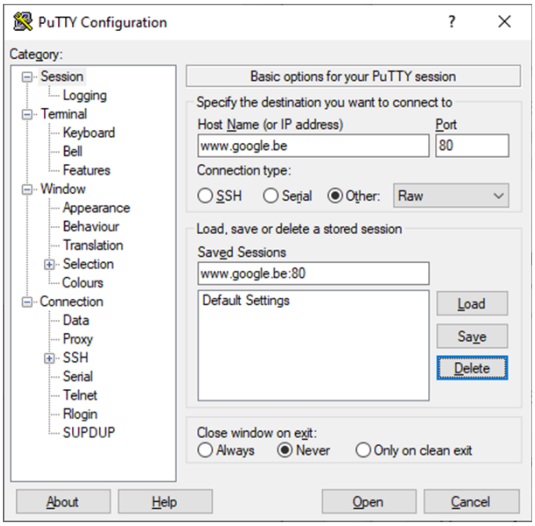
- **200 OK**
  - Request succeeded and information is returned in response
- **301 Moved Permanently**
  - Requested object has been permanently moved to a new URL (specified via the Location: header in the response)
- **400 Bad Request**
  - Generic error code indicating the server could not understand the request
- **404 Not Found**
  - The requested document does not exist on the server
- **505 HTTP Version Not Supported**
  - The requested HTTP protocol version is not supported by the server

The status code and associated phrase indicate the result of the request. Some common status codes and associated phrases include:

- 200 OK: Request succeeded and the information is returned in the response.
- 301 Moved Permanently: Requested object has been permanently moved; the new URL is specified in Location: header of the response message. The client software will automatically retrieve the new URL.
- 400 Bad Request: This is a generic error code indicating that the request could not be understood by the server.
- 404 Not Found: The requested document does not exist on this server.
- 505 HTTP Version Not Supported: The requested HTTP protocol version is not supported by the server.

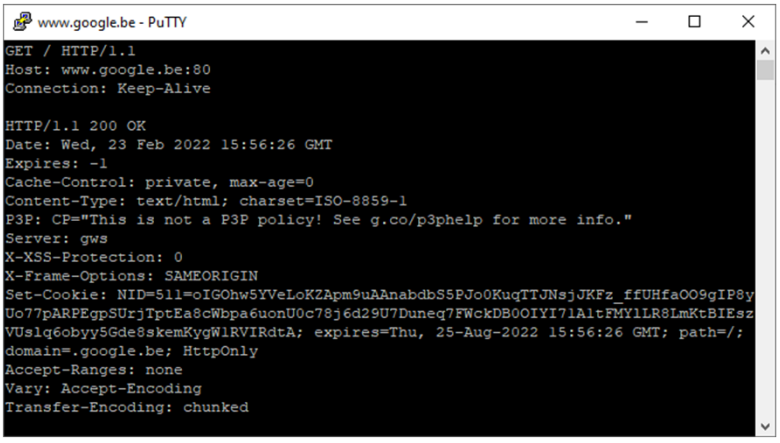
Homework: hands-on testing

Open a raw TCP connection (e.g., using PuTTY):



University of Antwerp
Faculty of Science

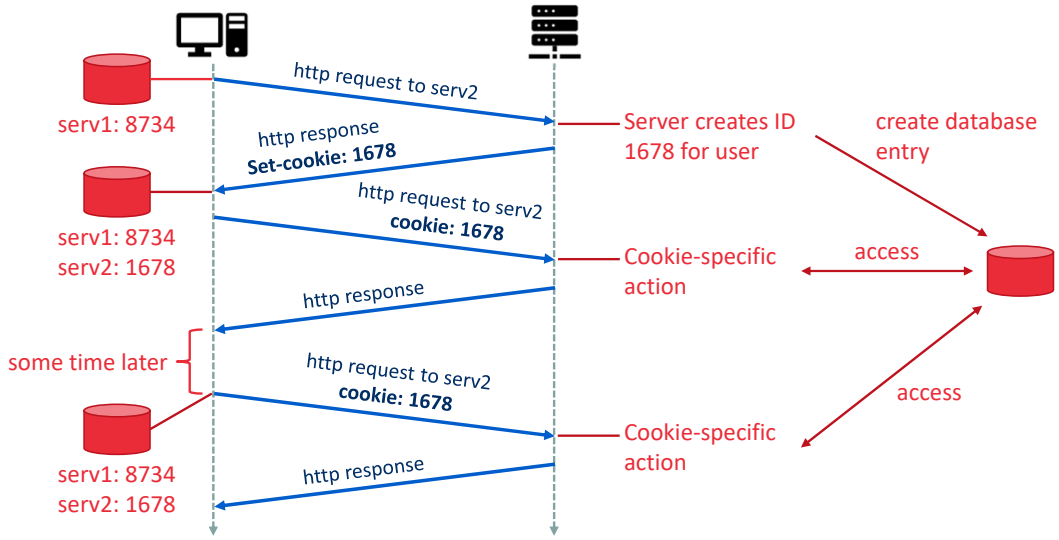
Request the main page ("/") of www.google.be at port 80:



Homework: Try this out yourself, while capturing TCP traffic to or from port 80 with Wireshark

Tip: Use PuTTY (Windows), or netcat/nc (cross-platform) to open RAW TCP connection

How to keep state with stateless HTTP protocol? Cookies!



University of Antwerp
Faculty of Science

29

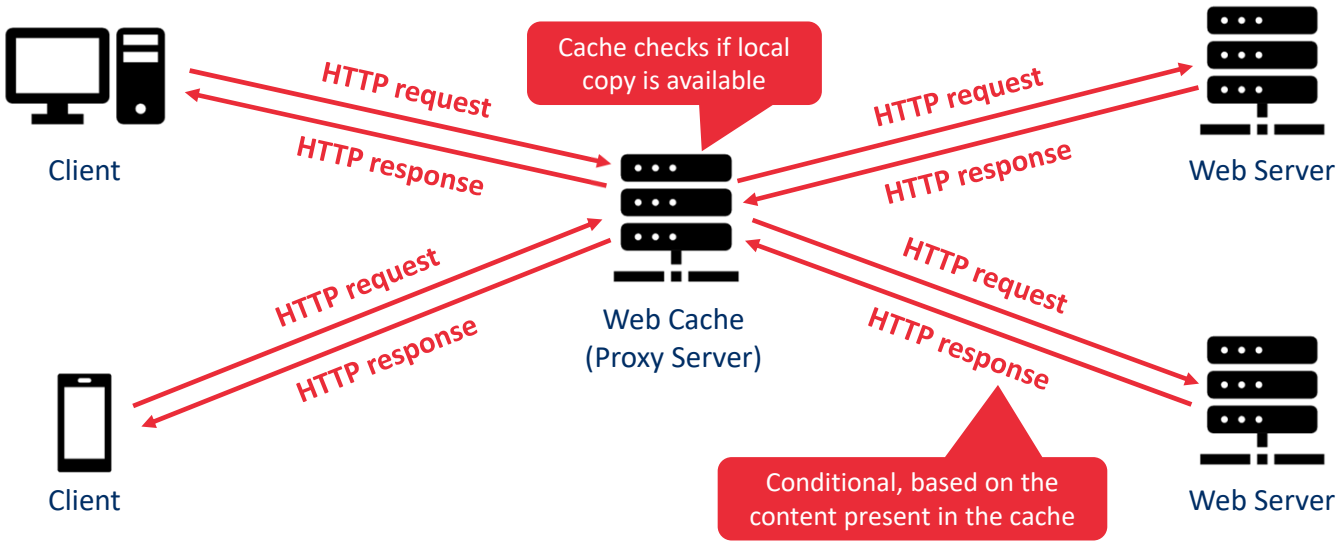
We mentioned before that an HTTP server is stateless. This simplifies server design and has permitted engineers to develop high-performance Web servers that can handle thousands of simultaneous TCP connections. However, it is often desirable for a Web site to identify users, either because the server wishes to restrict user access or because it wants to serve content as a function of the user identity. For these purposes, HTTP uses cookies. Cookies, defined in RFC 6265, allow sites to keep track of users. Most major commercial Web sites use cookies today.

As shown in the figure, cookie technology has four components: (1) a cookie header line in the HTTP response message; (2) a cookie header line in the HTTP request message; (3) a cookie file kept on the user's end system and managed by the user's browser; and (4) a back-end database at the Web site.

Cookies can be used to identify a user. The first time a user visits a site, the user can provide a user identification (possibly his or her name). During the subsequent sessions, the browser passes a cookie header to the server, thereby identifying the user to the server. Cookies can thus be used to create a user session layer on top of stateless HTTP. For example, when a user logs in to a Web-based e-mail application (such as Hotmail), the browser sends cookie information to the server, permitting the server to identify the user throughout the user's session with the application.

Although cookies often simplify the experience for the user, they are controversial because they can also be considered as an invasion of privacy. As we just saw, using a combination of cookies and user-supplied account information, a Web site can learn a lot about a user and potentially sell this information to a third party.

Web caches (also known as proxy servers)



University of Antwerp | Faculty of Science

30

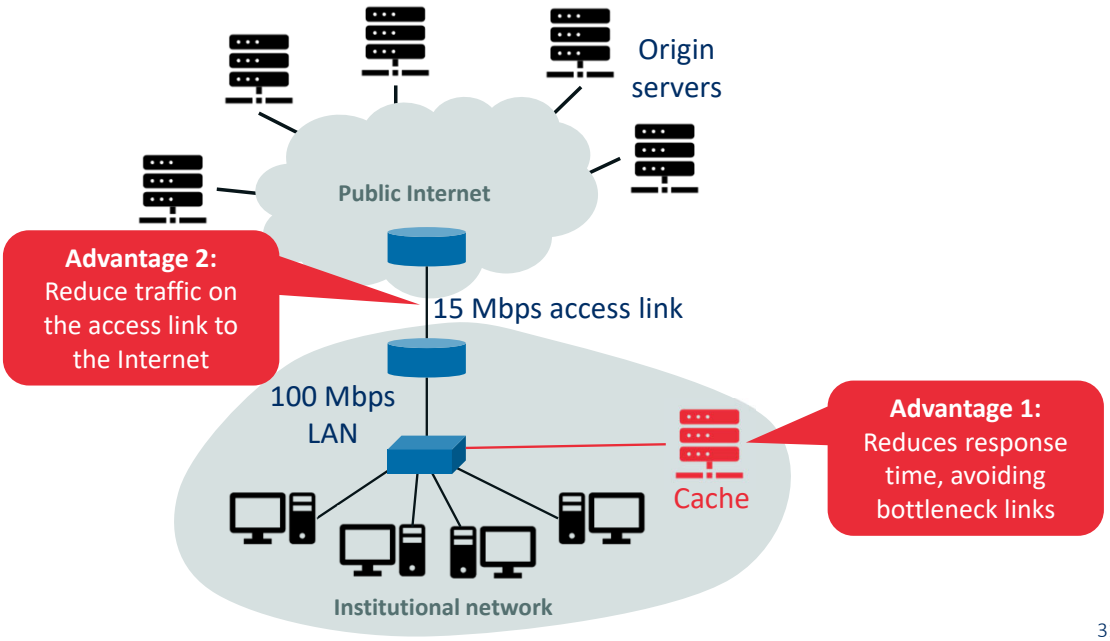
A **Web cache**—also called a **proxy server**—is a network entity that satisfies HTTP requests on the behalf of an origin Web server. The Web cache has its own disk storage and keeps copies of recently requested objects in this storage. As shown in the figure, a user's browser can be configured so that all of the user's HTTP requests are first directed to the Web cache (described in RFC 7234). Once a browser is configured, each browser request for an object is first directed to the Web cache.

As an example, suppose a browser is requesting the object http://www.someschool.edu/campus.gif. Here is what happens:

1. The browser establishes a TCP connection to the Web cache and sends an HTTP request for the object to the Web cache.
2. The Web cache checks to see if it has a copy of the object stored locally. If it does, the Web cache returns the object within an HTTP response message to the client browser.
3. If the Web cache does not have the object, the Web cache opens a TCP connection to the origin server, that is, to www.someschool.edu. The Web cache then sends an HTTP request for the object into the cache-to-server TCP connection. After receiving this request, the origin server sends the object within an HTTP response to the Web cache.
4. When the Web cache receives the object, it stores a copy in its local storage and sends a copy, within an HTTP response message, to the client browser (over the existing TCP connection between the client browser and the Web cache).

Note that a cache is both a server and a client at the same time. When it receives requests from and sends responses to a browser, it is a server. When it sends requests to and receives responses from an origin server, it is a client.

Advantages of caching



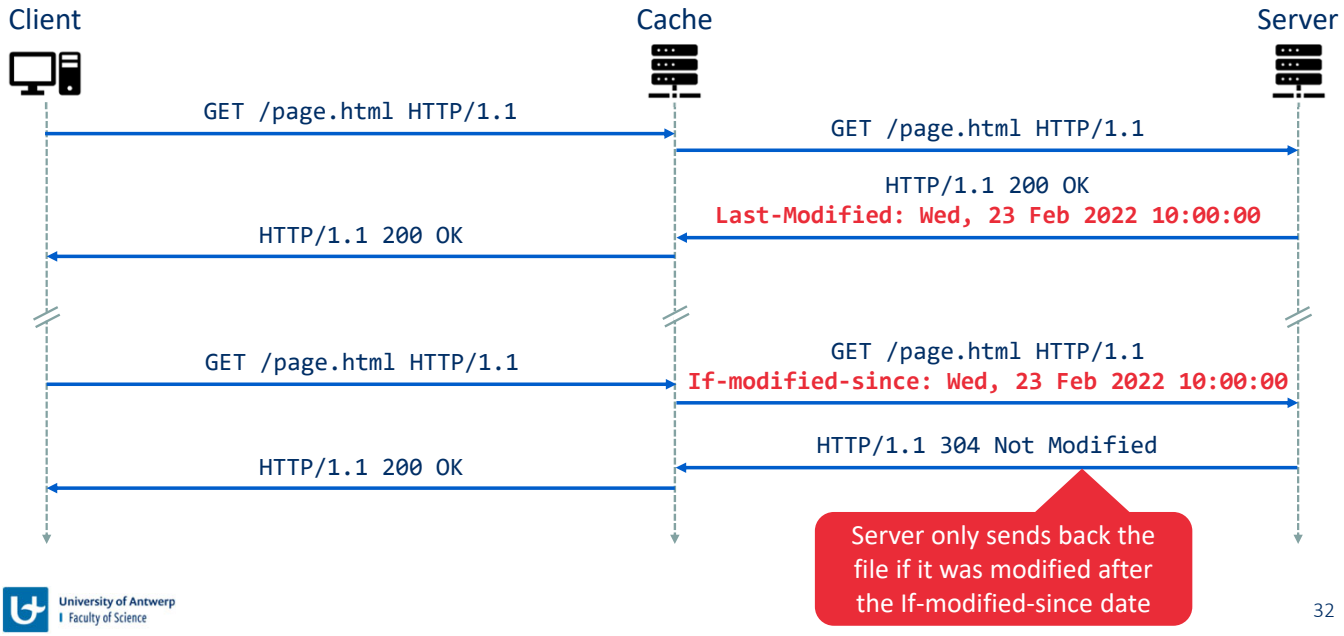
University of Antwerp
Faculty of Science

31

Web caching has seen deployment in the Internet for two reasons. First, a Web cache can substantially reduce the response time for a client request, particularly if the bottleneck bandwidth between the client and the origin server is much less than the bottleneck bandwidth between the client and the cache. If there is a high-speed connection between the client and the cache, as there often is, and if the cache has the requested object, then the cache will be able to deliver the object rapidly to the client. Second, Web caches can substantially reduce traffic on an institution's access link to the Internet. By reducing traffic, the institution (for example, a company or a university) does not have to upgrade bandwidth as quickly, thereby reducing costs. Furthermore, Web caches can substantially reduce Web traffic in the Internet as a whole, thereby improving performance for all applications.

Through the use of **Content Distribution Networks (CDNs)**, Web caches are increasingly playing an important role in the Internet. A CDN company installs many geographically distributed caches throughout the Internet, thereby localizing much of the traffic. There are shared CDNs (such as Akamai and Limelight) and dedicated CDNs (such as Google and Netflix).

How does the cache verify if its copy is still up to date? Conditional GET!



University of Antwerp | Faculty of Science

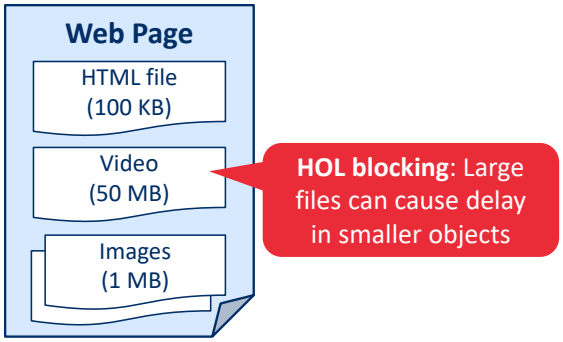
32

Although caching can reduce user-perceived response times, it introduces a new problem—the copy of an object residing in the cache may be stale. In other words, the object housed in the Web server may have been modified since the copy was cached at the client. Fortunately, HTTP has a mechanism that allows a cache to verify that its objects are up to date. This mechanism is called the **conditional GET** [RFC 7232]. An HTTP request message is a so-called conditional GET message if (1) the request message uses the GET method and (2) the request message includes an If-Modified-Since: header line.

To illustrate how the conditional GET operates, let's walk through an example. First, on the behalf of a requesting browser, a proxy cache sends a request message to a Web server. Second, the Web server sends a response message with the requested object to the cache. The cache forwards the object to the requesting browser but also caches the object locally. Importantly, the cache also stores the last-modified date along with the object. Third, one week later, another browser requests the same object via the cache, and the object is still in the cache. Since this object may have been modified at the Web server in the past week, the cache performs an up-to-date check by issuing a conditional GET. Note that the value of the If-modified-since: header line is exactly equal to the value of the Last-Modified: header line that was sent by the server one week ago. This conditional GET is telling the server to send the object only if the object has been modified since the specified date. Suppose the object has not been modified during this time period. Then, fourth, the Web server sends a response message with status 304 (Not Modified) to the cache. We see that in response to the conditional GET, the Web server still sends a response message but does not include the requested object in the response message. Including the requested object would only waste bandwidth and increase user-perceived response time, particularly if the object is large. The 304 status tells the cache that it can go ahead and forward its (the proxy cache's) cached copy of the object to the requesting browser.

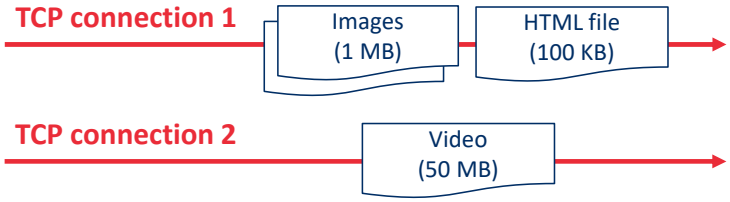
HTTP/2

- Does not change the HTTP methods, status codes, URLs or header fields over previous versions
- **Reduces perceived latency** through multiplexing, request prioritization, server push and compression
- Aims to solve **Head of Line (HOL) blocking** problem



HTTP/1.1 solution:

- Open multiple simultaneous TCP connections
- Interferes with TCP congestion control and fairness mechanisms



University of Antwerp
Faculty of Science

33

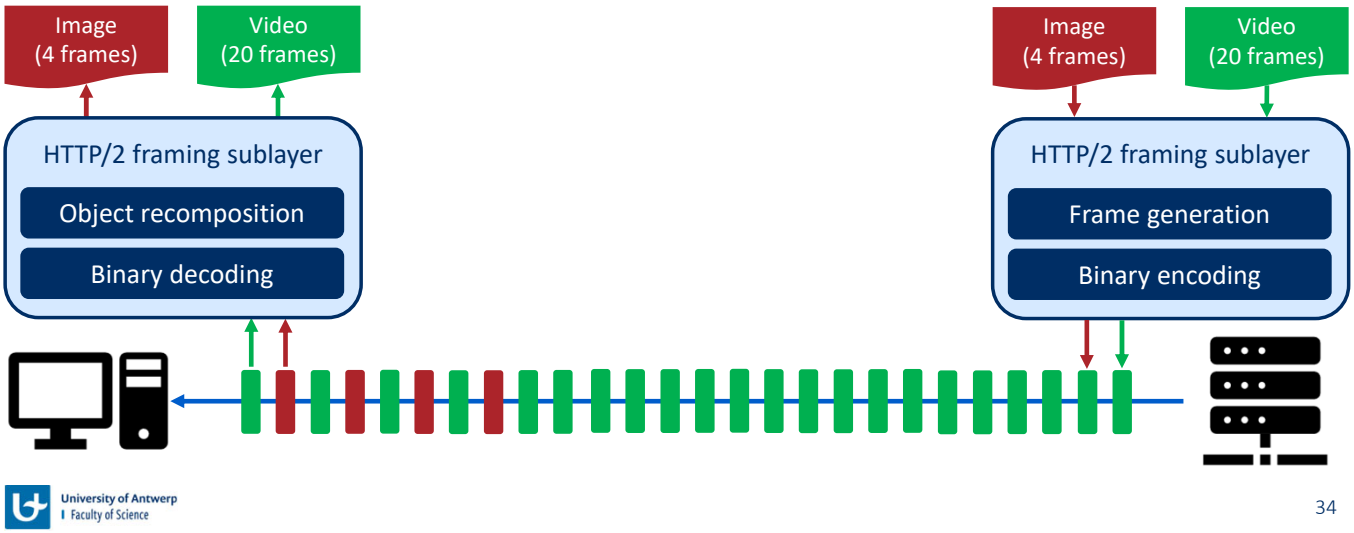
HTTP/2 [RFC 7540], standardized in 2015, was the first new version of HTTP since HTTP/1.1, which was standardized in 1997. Since standardization, HTTP/2 has taken off, with over 40% of the top 10 million websites supporting HTTP/2 in 2020. Most browsers—including Google Chrome, Internet Explorer, Safari, Opera, and Firefox—also support HTTP/2. The primary goals for HTTP/2 are to reduce perceived latency by enabling request and response multiplexing over a *single* TCP connection, provide request prioritization and server push, and provide efficient compression of HTTP header fields. HTTP/2 does not change HTTP methods, status codes, URLs, or header fields. Instead, HTTP/2 changes how the data is formatted and transported between the client and server.

Sending all the objects in a Web page over a single TCP connection has a **Head of Line (HOL) blocking** problem. To understand HOL blocking, consider a Web page that includes an HTML base page, a large video clip near the top of Web page, and many small objects below the video. Further suppose there is a low-to-medium speed bottleneck link (for example, a low-speed wireless link) on the path between server and client. Using a single TCP connection, the video clip will take a long time to pass through the bottleneck link, while the small objects are delayed as they wait behind the video clip; that is, the video clip at the head of the line blocks the small objects behind it. HTTP/1.1 browsers typically work around this problem by opening multiple parallel TCP connections, thereby having objects in the same web page sent in parallel to the browser.

One of the primary goals of HTTP/2 is to get rid of (or at least reduce the number of) parallel TCP connections for transporting a single Web page. This not only reduces the number of sockets that need to be open and maintained at servers, but also allows TCP congestion control to operate as intended. But with only one TCP connection to transport a Web page, HTTP/2 requires carefully designed mechanisms to avoid HOL blocking.

HTTP/2 HOL blocking solution: Framing

- Each message (object) is broken into small frames
- Request and response frames are interleaved over a single TCP connection



University of Antwerp
Faculty of Science

34

The HTTP/2 solution for HOL blocking is to break each message into small frames, and interleave the request and response messages on the same TCP connection. To understand this, consider again the example of a Web page consisting of one large video clip and, say, one smaller image object. Thus the server will receive 2 concurrent requests from any browser wanting to see this Web page. For each of these requests, the server needs to send 2 competing HTTP response messages to the browser. Suppose all frames are of fixed length, the video clip consists of 20 frames, and the smaller object consists of 4 frames. With frame interleaving, after sending one frame from the video clip, the first frame of the small object is sent. Then after sending the fourth frame of the video clip, the last frame of the small object is sent. Thus, the smaller object is sent after sending a total of 8 frames. If interleaving were not used, the smaller object would be sent only after sending 24 frames. Thus the HTTP/2 framing mechanism can significantly decrease user-perceived delay.

The ability to break down an HTTP message into independent frames, interleave them, and then reassemble them on the other end is the single most important enhancement of HTTP/2. The framing is done by the framing sub-layer of the HTTP/2 protocol. When a server wants to send an HTTP response, the response is processed by the framing sub-layer, where it is broken down into frames. The header field of the response becomes one frame, and the body of the message is broken down into one or more additional frames. The frames of the response are then interleaved by the framing sub-layer in the server with the frames of other responses and sent over the single persistent TCP connection. As the frames arrive at the client, they are first reassembled into the original response messages at the framing sub-layer and then processed by the browser as usual. Similarly, a client's HTTP requests are broken into frames and interleaved. In addition to breaking down each HTTP message into independent frames, the framing sublayer also binary encodes the frames. Binary protocols are more efficient to parse, lead to slightly smaller frames, and are less error-prone.

Other HTTP/2 features

- **Message Prioritization**
  - Allows web developers to customize the relative priority of requests
  - Each request is assigned a weight between 1 (lowest) and 256 (highest)
  - The server first sends the frames of responses with highest priority (interleaving those with equal priority)
  - Client can define dependencies between requests
- **Server Push**
  - Allows the server to send multiple responses to a single request
  - Useful to pro-actively send additional content that is part of a requested HTML base page
  - Can also be used for pushing segmented video
  - Eliminates the extra latency due to waiting for multiple requests

Message prioritization allows developers to customize the relative priority of requests to better optimize application performance. As we just learned, the framing sub-layer organizes messages into parallel streams of data destined to the same requestor. When a client sends concurrent requests to a server, it can prioritize the responses it is requesting by assigning a weight between 1 and 256 to each message. The higher number indicates higher priority. Using these weights, the server can send first the frames for the responses with the highest priority. In addition to this, the client also states each message's dependency on other messages by specifying the ID of the message on which it depends.

Another feature of HTTP/2 is the ability for a server to send multiple responses for a single client request. That is, in addition to the response to the original request, the server can *push* additional objects to the client, without the client having to request each one. This is possible since the HTML base page indicates the objects that will be needed to fully render the Web page. So instead of waiting for the HTTP requests for these objects, the server can analyze the HTML page, identify the objects that are needed, and send them to the client *before receiving explicit requests for these objects*. Server push eliminates the extra latency due to waiting for the requests.

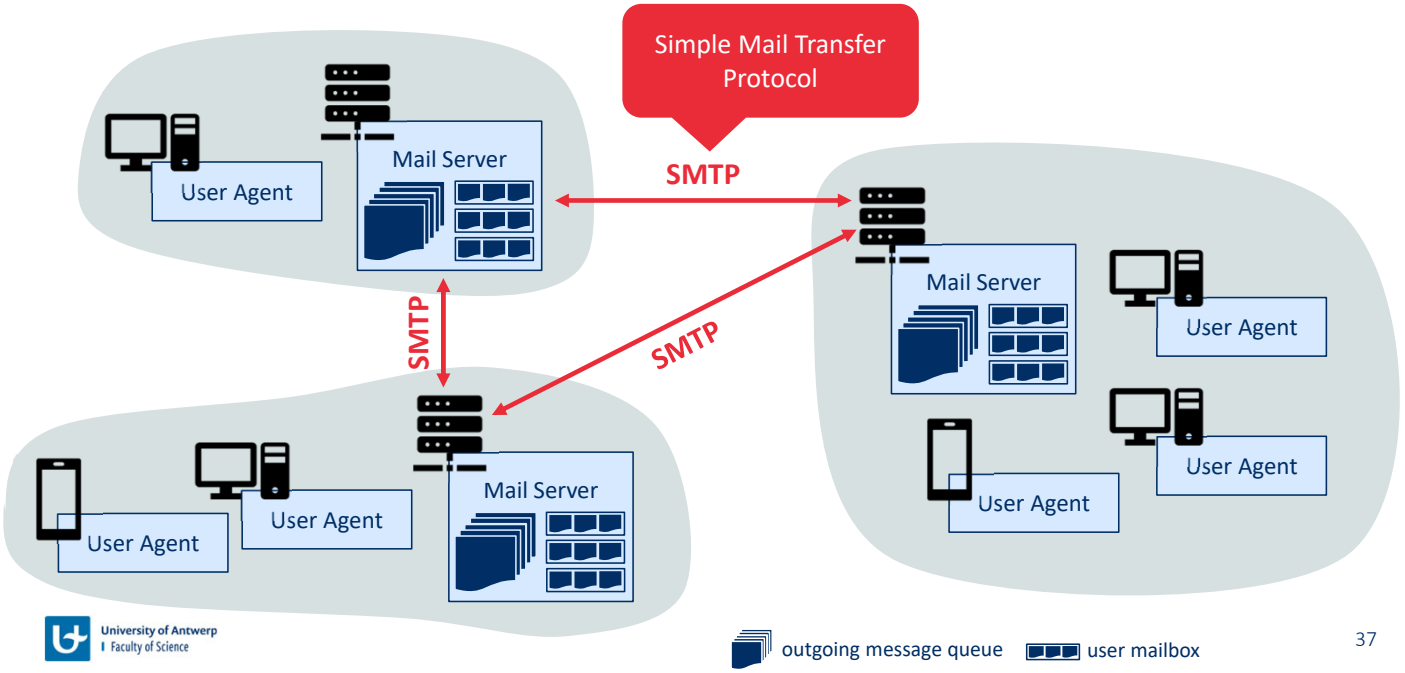
Electronic mail (email)

University of Antwerp
Faculty of Science

Section 2.3

Electronic mail has been around since the beginning of the Internet. It was the most popular application when the Internet was in its infancy, and has become more elaborate and powerful over the years. It remains one of the Internet's most important and utilized applications. As with ordinary postal mail, e-mail is an asynchronous communication medium—people send and read messages when it is convenient for them, without having to coordinate with other people's schedules. In contrast with postal mail, electronic mail is fast, easy to distribute, and inexpensive. Modern e-mail has many powerful features, including messages with attachments, hyperlinks, HTML-formatted text, and embedded photos.

High-level overview of Internet email



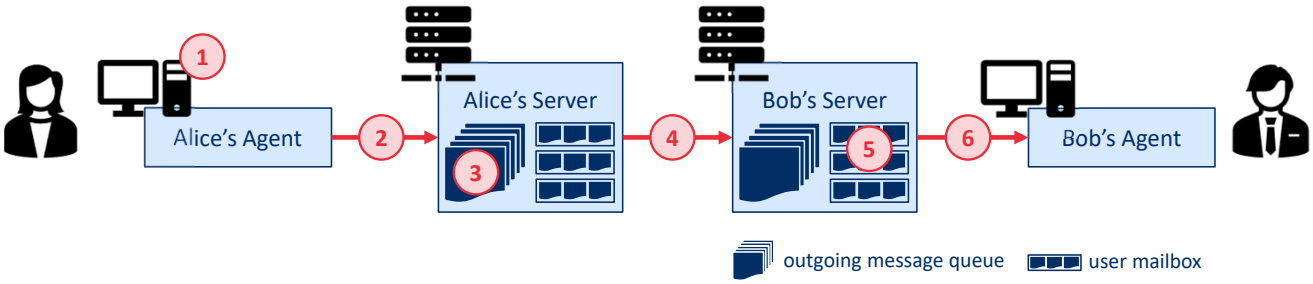
We see from this diagram that it has three major components: **user agents**, **mail servers**, and the **Simple Mail Transfer Protocol (SMTP)**. We now describe each of these components in the context of a sender, Alice, sending an e-mail message to a recipient, Bob. User agents allow users to read, reply to, forward, save, and compose messages. Examples of user agents for e-mail include Microsoft Outlook, Apple Mail, Web-based Gmail, the Gmail App running in a smartphone, and so on. When Alice is finished composing her message, her user agent sends the message to her mail server, where the message is placed in the mail server's outgoing message queue. When Bob wants to read a message, his user agent retrieves the message from his mailbox in his mail server.

Mail servers form the core of the e-mail infrastructure. Each recipient, such as Bob, has a **mailbox** located in one of the mail servers. Bob's mailbox manages and maintains the messages that have been sent to him. A typical message starts its journey in the sender's user agent, then travels to the sender's mail server, and then travels to the recipient's mail server, where it is deposited in the recipient's mailbox. When Bob wants to access the messages in his mailbox, the mail server containing his mailbox authenticates Bob (with his username and password). Alice's mail server must also deal with failures in Bob's mail server. If Alice's server cannot deliver mail to Bob's server, Alice's server holds the message in a **message queue** and attempts to transfer the message later. Reattempts are often done every 30 minutes or so; if there is no success after several days, the server removes the message and notifies the sender (Alice) with an e-mail message.

SMTP is the principal application-layer protocol for Internet electronic mail. It uses the reliable data transfer service of TCP to transfer mail from the sender's mail server to the recipient's mail server. As with most application-layer protocols, SMTP has two sides: a client side, which executes on the sender's mail server, and a server side, which executes on the recipient's mail

server. Both the client and server sides of SMTP run on every mail server. When a mail server sends mail to other mail servers, it acts as an SMTP client. When a mail server receives mail from other mail servers, it acts as an SMTP server.

Simple Mail Transfer Protocol (SMTP) – RFC 5321



1. Alice composes an email to bob@emailprovider.com and instructs her users agent to send it
2. Alice's user agent sends the message to her mail server, where it is placed in the message queue
3. The SMTP client on Alice's mail server opens a TCP connection to the SMTP server on Bob's mail server
4. After the SMTP handshake, the SMTP client sends Alice's message via TCP
5. The SMTP server on Bob's mail server receives the message and places it into Bob's mailbox
6. Bob invokes his user agent to read the message at his convenience

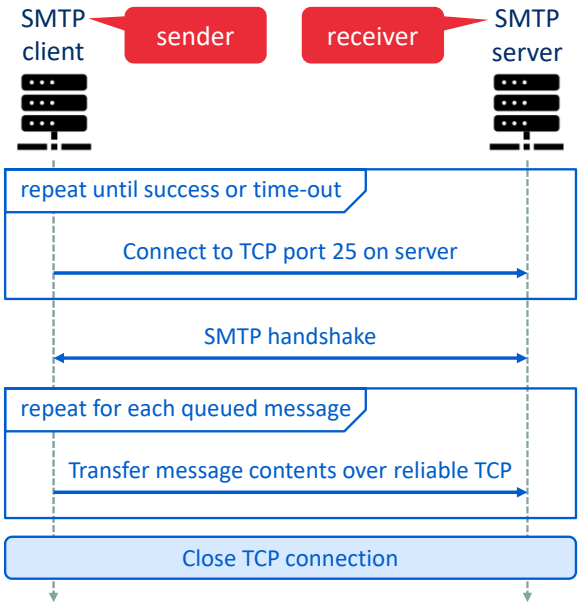
SMTP, defined in RFC 5321, is at the heart of Internet electronic mail. As mentioned above, SMTP transfers messages from senders' mail servers to the recipients' mail servers. SMTP is much older than HTTP. (The original SMTP RFC dates back to 1982, and SMTP was around long before that.) Although SMTP has numerous wonderful qualities, as evidenced by its ubiquity in the Internet, it is nevertheless a legacy technology that possesses certain archaic characteristics. For example, it restricts the body (not just the headers) of all mail messages to simple 7-bit ASCII. This restriction made sense in the early 1980s when transmission capacity was scarce and no one was e-mailing large attachments or large image, audio, or video files. But today, in the multimedia era, the 7-bit ASCII restriction is a bit of a pain—it requires binary multimedia data to be encoded to ASCII before being sent over SMTP; and it requires the corresponding ASCII message to be decoded back to binary after SMTP transport.

To illustrate the basic operation of SMTP, let's walk through a common scenario. Suppose Alice wants to send Bob a simple ASCII message:

1. Alice invokes her user agent for e-mail, provides Bob's e-mail address (for example, bob@emailprovider.com), composes a message, and instructs the user agent to send the message.
2. Alice's user agent sends the message to her mail server, where it is placed in a message queue.
3. The client side of SMTP, running on Alice's mail server, sees the message in the message queue. It opens a TCP connection to an SMTP server, running on Bob's mail server.
4. After some initial SMTP handshaking, the SMTP client sends Alice's message into the TCP connection.
5. At Bob's mail server, the server side of SMTP receives the message. Bob's mail server then places the message in Bob's mailbox.
6. Bob invokes his user agent to read the message at his convenience.

It is important to observe that SMTP does not normally use intermediate mail servers for sending mail, even when the two mail servers are located at opposite ends of the world. If Alice's server is in Hong Kong and Bob's server is in St. Louis, the TCP connection is a direct connection between the Hong Kong and St. Louis servers. In particular, if Bob's mail server is down, the message remains in Alice's mail server and waits for a new attempt—the message does not get placed in some intermediate mail server.

Details of SMTP message transfer



University of Antwerp
Faculty of Science

Example

From client (C) crepes.fr to server (S) hamburger.edu:

Handshake

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
```

Message

```
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
```

Close connection

```
C: QUIT
S: 221 hamburger.edu closing connection
```

39

Let's now take a closer look at how SMTP transfers a message from a sending mail server to a receiving mail server. We will see that the SMTP protocol has many similarities with protocols that are used for face-to-face human interaction. First, the client SMTP (running on the sending mail server host) has TCP establish a connection to port 25 at the server SMTP (running on the receiving mail server host). If the server is down, the client tries again later. Once this connection is established, the server and client perform some application-layer handshaking—just as humans often introduce themselves before transferring information from one to another, SMTP clients and servers introduce themselves before transferring information. During this SMTP handshaking phase, the SMTP client indicates the e-mail address of the sender (the person who generated the message) and the e-mail address of the recipient. Once the SMTP client and server have introduced themselves to each other, the client sends the message. SMTP can count on the reliable data transfer service of TCP to get the message to the server without errors. The client then repeats this process over the same TCP connection if it has other messages to send to the server; otherwise, it instructs TCP to close the connection.

Let's next take a look at an example transcript of messages exchanged between an SMTP client (C) and an SMTP server (S). The hostname of the client is crepes.fr and the hostname of the server is hamburger.edu. The ASCII text lines prefaced with C: are exactly the lines the client sends into its TCP socket, and the ASCII text lines prefaced with S: are exactly the lines the server sends into its TCP socket. In the example, the client sends a message ("Do you like ketchup? How about pickles?") from mail server crepes.fr to mail server hamburger.edu. As part of the dialogue, the client issued five commands: HELO (an abbreviation for HELLO), MAIL FROM, RCPT TO, DATA, and QUIT. These commands are self-explanatory. The client also sends a line consisting of a single period, which indicates the end of the message to the server. (In ASCII jargon, each message ends with CRLF.CRLF, where CR and LF stand for carriage return and line

feed, respectively.) The server issues replies to each command, with each reply having a reply code and some (optional) English-language explanation. We mention here that SMTP uses persistent connections: If the sending mail server has several messages to send to the same receiving mail server, it can send all of the messages over the same TCP connection. For each message, the client begins the process with a new MAIL FROM: crepes.fr, designates the end of message with an isolated period, and issues QUIT only after all messages have been sent.

Internet Message Format – RFC 5322

- Represents the email contents (i.e., the DATA part in the sent SMTP message)
- Uses 7-bit ASCII encoding (like all SMTP messages)
- Consists of header fields (standardized in RFC 5322) and a message body
  - Header fields consist of a keyword, colon, and value
  - Header and body are separated by a blank line
- Example

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Searching for the meaning of life.

MESSAGE DATA...
```


When Alice writes an ordinary snail-mail letter to Bob, she may include all kinds of peripheral header information at the top of the letter, such as Bob's address, her own return address, and the date. Similarly, when an e-mail message is sent from one person to another, a header containing peripheral information precedes the body of the message itself. This peripheral information is contained in a series of header lines, which are defined in RFC 5322. The header lines and the body of the message are separated by a blank line (that is, by CRLF). RFC 5322 specifies the exact format for mail header lines as well as their semantic interpretations. As with HTTP, each header line contains readable text, consisting of a keyword followed by a colon followed by a value. Some of the keywords are required and others are optional. Every header must have a From: header line and a To: header line; a header may include a Subject: header line as well as other optional header lines. It is important to note that these header lines are *different* from the SMTP commands (even though they contain some common words such as "*from*" and "*to*"). The commands were part of the SMTP handshaking protocol; the header lines examined here are part of the mail message itself.

Multipurpose Internet Mail Extensions (MIME) – RFCs 2045 and 2046

Type	Subtypes	Description
Text	Plain, enriched	Unformatted for formatted text
Multipart	Mixed, parallel, alternative, digest	Content of multiple types transmitted together
Message	Rfc822, partial, external-body	RFC822 compliant, large fragmented message, or pointer to external object
Image	Jpeg, gif	Image in JPEG or GIF format
Video	Mpeg	Video in MPEG format
Audio	Basic	Audio encoded at 8 kHz
Application	Postscript, octet-stream	Postscript file or general binary data

Example

```
From: Nathaniel Borenstein <nsb@bellcore.com>
To: Ned Freed <ned@innosoft.com>
Subject: Sample message
MIME-Version: 1.0
Content-type: multipart/mixed; boundary="simple boundary"

This is the preamble. It is to be ignored, though it is a handy
place for mail composers to include an explanatory note to non-
MIME conformant readers.
--simple boundary

This is implicitly typed plain ASCII text. It does NOT end with
a linebreak.
--simple boundary
Content-type: text/plain; charset=us-ascii

This is explicitly typed plain ASCII text. It DOES end with a
linebreak.

--simple boundary--
This is the epilogue. It is also to be ignored.
```

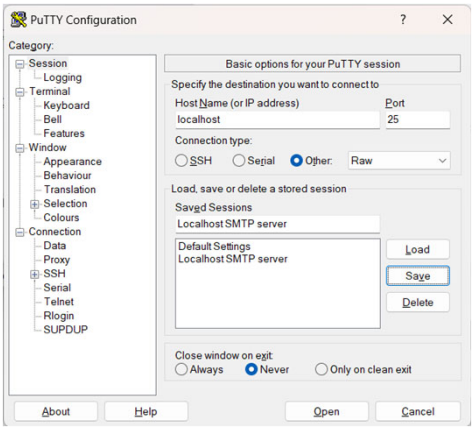

The bulk of the MIME specification is concerned with the definition of a variety of content types. This reflects the need to provide standardized ways of dealing with a wide variety of information representations in a multimedia environment. The table lists the content types specified in RFCs 2045 and 2046. There are seven different major types of content and a total of 15 subtypes. In general, a content type declares the general type of data, and the subtype specifies a particular format for that type of data.

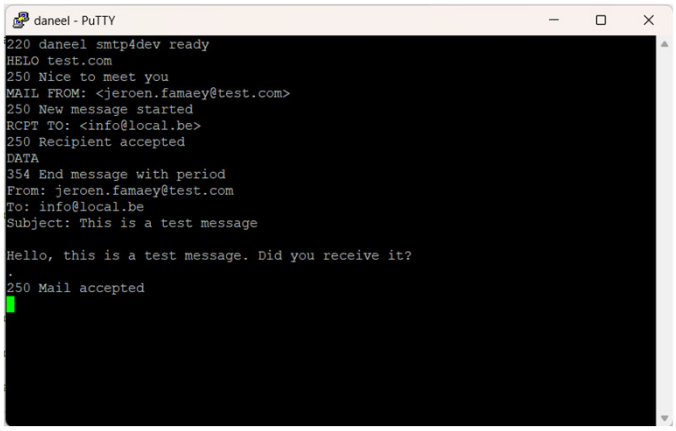
For the **text type** of body, no special software is required to get the full meaning of the text aside from support of the indicated character set. The primary subtype is *plain text*, which is simply a string of ASCII characters or ISO 8859 characters. The *enriched* subtype allows greater formatting flexibility.

The **multipart type** indicates that the body contains multiple, independent parts. The Content-Type header field includes a parameter (called boundary) that defines the delimiter between body parts. This boundary should not appear in any parts of the message. Each boundary starts on a new line and consists of two hyphens followed by the boundary value. The final boundary, which indicates the end of the last part, also has a suffix of two hyphens. Within each part, there may be an optional ordinary MIME header.

Homework: Testing out SMTP

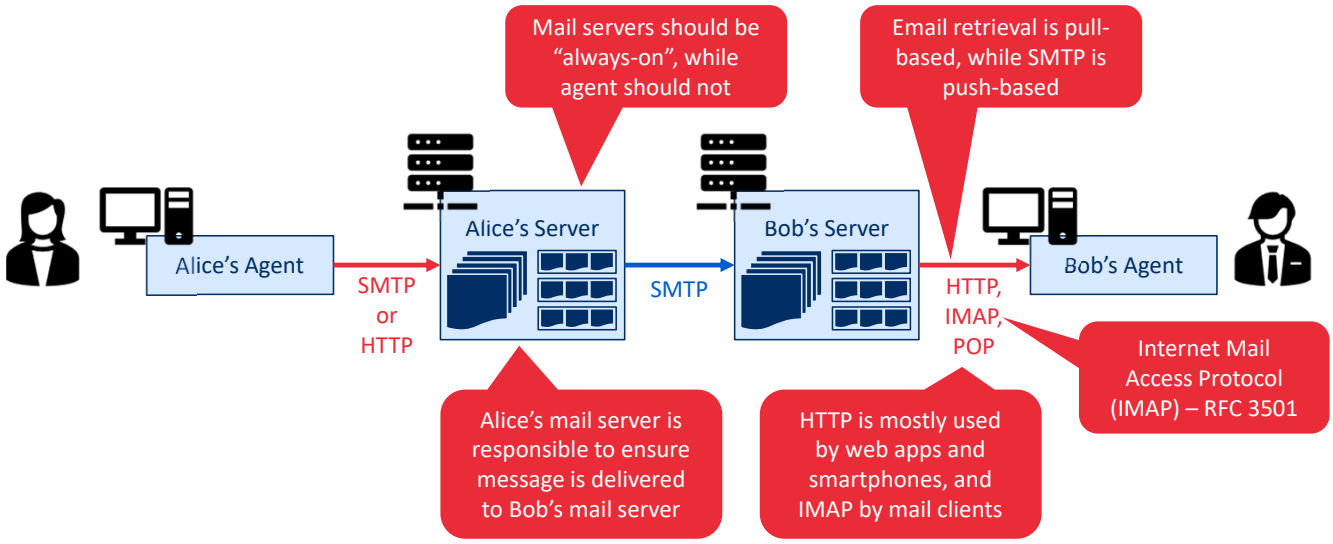
- Easiest way to test SMTP is to install a local SMTP server (e.g., https://github.com/rnwood/smtp4dev, or MailHog)





It is highly recommended that you use Telnet to carry out a direct dialogue with an SMTP server. To do this, issue “telnet serverName 25”, where serverName is the name of a local mail server (e.g., out.telenet.be). When you do this, you are simply establishing a TCP connection between your local host and the mail server. After typing this line, you should immediately receive the 220 reply from the server. Then issue the SMTP commands HELO, MAIL FROM, RCPT TO, DATA, CRLF.CRLF, and QUIT at the appropriate times.

Mail access protocols (i.e., how to access your mailbox?)



Once SMTP delivers the message from Alice's mail server to Bob's mail server, the message is placed in Bob's mailbox. Given that Bob (the recipient) executes his user agent on his local host (e.g., smartphone or PC), it is natural to consider placing a mail server on his local host as well. With this approach, Alice's mail server would dialogue directly with Bob's PC. There is a problem with this approach, however. Recall that a mail server manages mailboxes and runs the client and server sides of SMTP. If Bob's mail server were to reside on his local host, then Bob's host would have to remain always on, and connected to the Internet, in order to receive new mail, which can arrive at any time. This is impractical for many Internet users. Instead, a typical user runs a user agent on the local host but accesses its mailbox stored on an always-on shared mail server. This mail server is shared with other users.

Alice's user agent uses SMTP or HTTP to deliver the e-mail message into her mail server, then Alice's mail server uses SMTP (as an SMTP client) to relay the e-mail message to Bob's mail server. Why the two-step procedure? Primarily because without relaying through Alice's mail server, Alice's user agent doesn't have any recourse to an unreachable destination mail server. By having Alice first deposit the e-mail in her own mail server, Alice's mail server can repeatedly try to send the message to Bob's mail server, say every 30 minutes, until Bob's mail server becomes operational.

But there is still one missing piece to the puzzle! How does a recipient like Bob, running a user agent on his local host , obtain his messages, which are sitting in a mail server? Note that Bob's user agent can't use SMTP to obtain the messages because obtaining the messages is a pull operation, whereas SMTP is a push protocol.

Today, there are two common ways for Bob to retrieve his e-mail from a mail server. If Bob is using Web-based e-mail or a smartphone app (such as Gmail), then the user agent will use HTTP

to retrieve Bob's e-mail. This case requires Bob's mail server to have an HTTP interface as well as an SMTP interface (to communicate with Alice's mail server). The alternative method, typically used with mail clients such as Microsoft Outlook, is to use the **Internet Mail Access Protocol (IMAP)** defined in RFC 3501. Both the HTTP and IMAP approaches allow Bob to manage folders, maintained in Bob's mail server. Bob can move messages into the folders he creates, delete messages, mark messages as important, and so on.

Domain Name System (DNS)

The Internet's Directory Service

University of Antwerp
Faculty of Science

Computer Networking
A Top-Down Approach

Section 2.4

Just as humans can be identified in many ways, so too can Internet hosts. One identifier for a host is its **hostname**. Hostnames—such as www.facebook.com, www.google.com, gaia.cs.umass.edu—are mnemonic and are therefore appreciated by humans. However, hostnames provide little, if any, information about the location within the Internet of the host. (A hostname such as www.eurecom.fr, which ends with the country code .fr, tells us that the host is probably in France, but doesn't say much more.) Furthermore, because hostnames can consist of variable-length alphanumeric characters, they would be difficult to process by routers. For these reasons, hosts are also identified by so-called **IP addresses**.

We discuss IP addresses in some detail later, but it is useful to say a few brief words about them now. An IP address consists of four bytes and has a rigid hierarchical structure. An IP address looks like 121.7.106.83, where each period separates one of the bytes expressed in decimal notation from 0 to 255. An IP address is hierarchical because as we scan the address from left to right, we obtain more and more specific information about where the host is located in the Internet (that is, within which network, in the network of networks). Similarly, when we scan a postal address from bottom to top, we obtain more and more specific information about where the addressee is located.

Domain name system (DNS) in a nutshell – RFCs 1034 and 1035

- Translates (human readable) hostnames to (router readable) IP addresses
- Consists of
  - A distributed hierarchy of DNS servers
  - An application-layer protocol that allows hosts to query the distributed database
- Makes use of UDP port 53
- Other services offered by DNS
  - **Host aliasing:** Defines a set of alternative (more mnemonic) hostnames in addition to the main **canonical hostname**
  - **Mail server aliasing:** Similar to host aliasing but for email addresses
  - **Load distribution:** Associating multiple IP addresses with 1 alias hostname to spread out traffic

University of Antwerp
Faculty of Science

We have just seen that there are two ways to identify a host—by a hostname and by an IP address. People prefer the more mnemonic hostname identifier, while routers prefer fixed-length, hierarchically structured IP addresses. In order to reconcile these preferences, we need a directory service that translates hostnames to IP addresses. This is the main task of the Internet's **domain name system (DNS)**. The DNS is (1) a distributed database implemented in a hierarchy of **DNS servers**, and (2) an application-layer protocol that allows hosts to query the distributed database. The DNS servers are often UNIX machines running the Berkeley Internet Name Domain (BIND) software. The DNS protocol runs over UDP and uses port 53. DNS is commonly employed by other application-layer protocols, including HTTP and SMTP, to translate user-supplied hostnames to IP addresses. DNS adds an additional delay—sometimes substantial—to the Internet applications that use it. Fortunately, as we discuss later, the desired IP address is often cached in a "nearby" DNS server, which helps to reduce DNS network traffic as well as the average DNS delay.

DNS provides a few other important services in addition to translating hostnames to IP addresses:

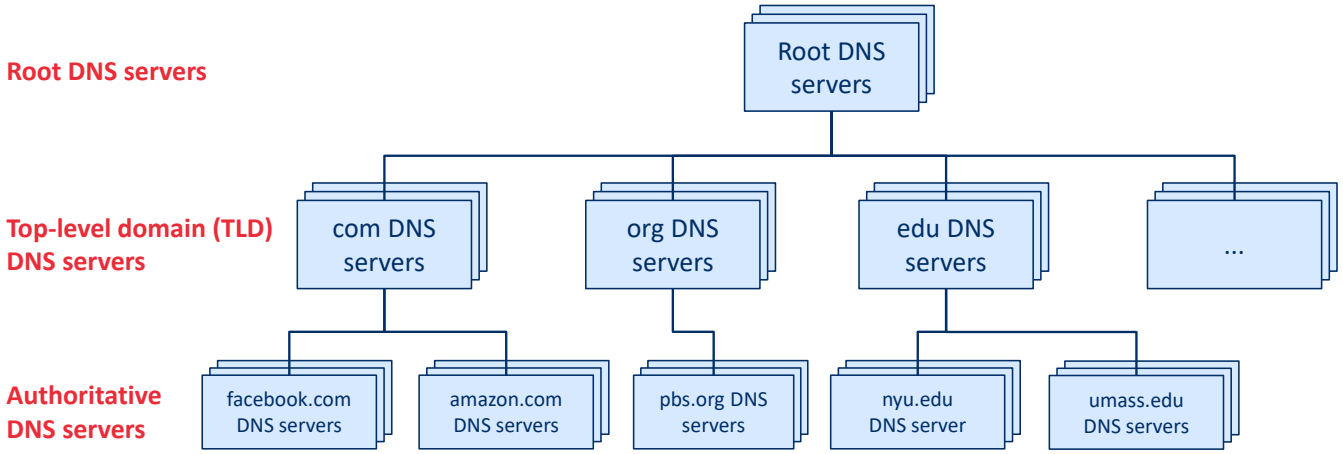
- **Host aliasing.** A host with a complicated hostname can have one or more alias names. For example, a hostname such as relay1.west-coast.enterprise.com could have, say, two aliases such as enterprise.com and www.enterprise.com. In this case, the hostname relay1.west-coast.enterprise.com is said to be a **canonical hostname**. Alias hostnames, when present, are typically more mnemonic than canonical hostnames. DNS can be invoked by an application to obtain the canonical hostname for a supplied alias hostname as well as the IP address of the host.
- **Mail server aliasing.** For obvious reasons, it is highly desirable that e-mail addresses be mnemonic. For example, if Bob has an account with Yahoo Mail, Bob's e-mail address might be as simple as bob@yahoo.com. However, the hostname of the Yahoo mail server is more

complicated and much less mnemonic than simply yahoo.com (for example, the canonical hostname might be something like relay1.west-coast.yahoo.com). DNS can be invoked by a mail application to obtain the canonical hostname for a supplied alias hostname as well as the IP address of the host. In fact, the MX record (see below) permits a company's mail server and Web server to have identical (aliased) hostnames; for example, a company's Web server and mail server can both be called enterprise.com.

- **Load distribution.** DNS is also used to perform load distribution among replicated servers, such as replicated Web servers. Busy sites, such as cnn.com, are replicated over multiple servers, with each server running on a different end system and each having a different IP address. For replicated Web servers, a *set* of IP addresses is thus associated with one alias hostname. The DNS database contains this set of IP addresses. When clients make a DNS query for a name mapped to a set of addresses, the server responds with the entire set of IP addresses, but rotates the ordering of the addresses within each reply. Because a client typically sends its HTTP request message to the IP address that is listed first in the set, DNS rotation distributes the traffic among the replicated servers. DNS rotation is also used for e-mail so that multiple mail servers can have the same alias name.

The DNS is specified in RFC 1034 and RFC 1035, and updated in several additional RFCs. It is a complex system, and we only touch upon key aspects of its operation here.

DNS is distributed in a hierarchical database



University of Antwerp
Faculty of Science

46

In order to deal with the issue of scale, the DNS uses a large number of servers, organized in a hierarchical fashion and distributed around the world. No single DNS server has all of the mappings for all of the hosts in the Internet. Instead, the mappings are distributed across the DNS servers. To a first approximation, there are three classes of DNS servers—root DNS servers, top-level domain (TLD) DNS servers, and authoritative DNS servers—organized in a hierarchy.

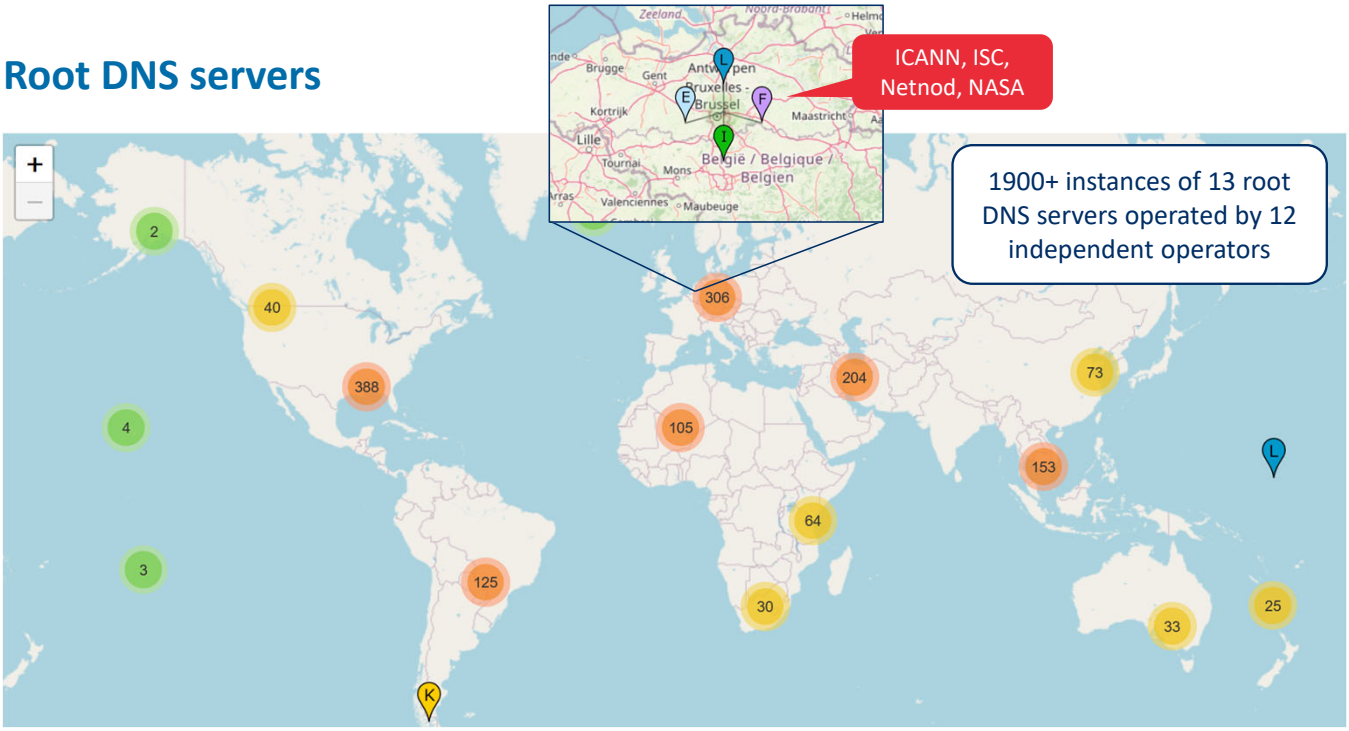
To understand how these three classes of servers interact, suppose a DNS client wants to determine the IP address for the hostname www.amazon.com. To a first approximation, the following events will take place. The client first contacts one of the root servers, which returns IP addresses for TLD servers for the top-level domain com. The client then contacts one of these TLD servers, which returns the IP address of an authoritative server for amazon.com. Finally, the client contacts one of the authoritative servers for amazon.com, which returns the IP address for the hostname www.amazon.com.

Let's take a closer look at these three classes of DNS servers:

- **Root DNS servers.** There are more than 1000 root servers instances scattered all over the world. These root servers are copies of 13 different root servers, managed by 12 different organizations, and coordinated through the Internet Assigned Numbers Authority. Root name servers provide the IP addresses of the TLD servers.
- **Top-level domain (TLD) servers.** For each of the top-level domains—top-level domains such as com, org, net, edu, and gov, and all of the country top-level domains such as uk, fr, ca, and jp—there is TLD server (or server cluster). The company Verisign Global Registry Services maintains the TLD servers for the com top-level domain, and the company Educause maintains the TLD servers for the edu top-level domain. The network infrastructure supporting a TLD can be large and complex. TLD servers provide the IP addresses for authoritative DNS servers.

- **Authoritative DNS servers.** Every organization with publicly accessible hosts (such as Web servers and mail servers) on the Internet must provide publicly accessible DNS records that map the names of those hosts to IP addresses. An organization's authoritative DNS server houses these DNS records. An organization can choose to implement its own authoritative DNS server to hold these records; alternatively, the organization can pay to have these records stored in an authoritative DNS server of some service provider. Most universities and large companies implement and maintain their own primary and secondary (backup) authoritative DNS server.

Root DNS servers



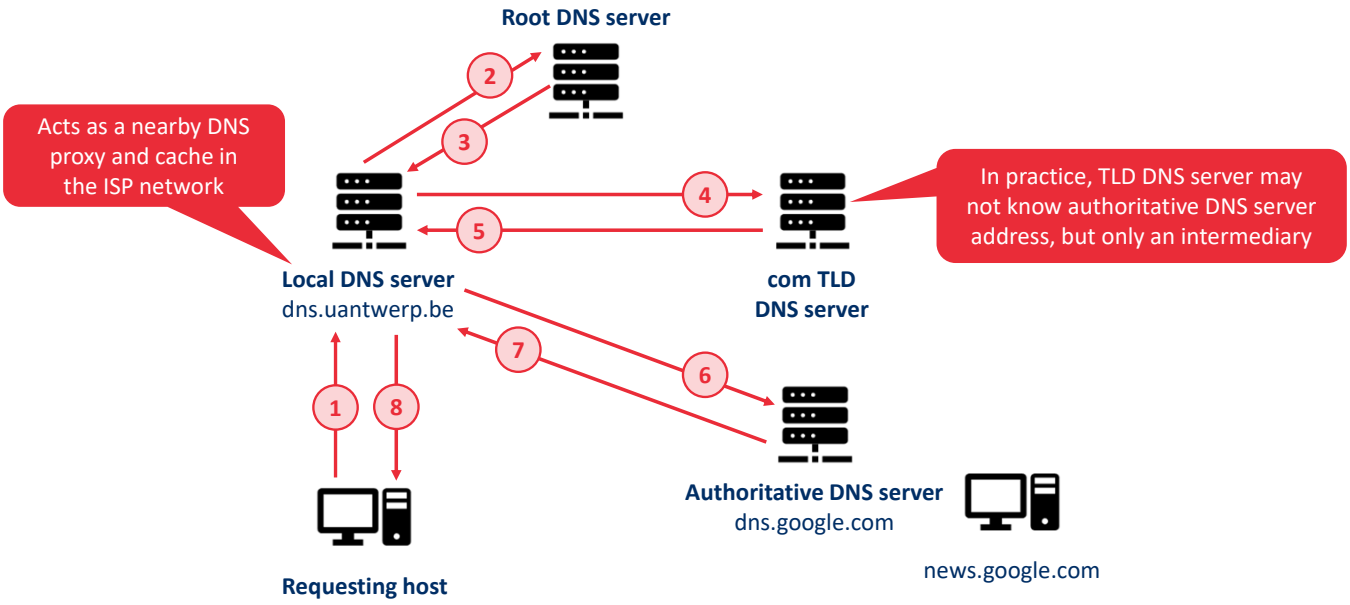
University of Antwerp | Faculty of Science

Dynamic map available at https://root-servers.org/, Last checked 18/2/2026

47

Currently there are 13 root DNS servers managed by 12 root server operators (i.e., Versign, USC-ISI, Cogent, UMD, NASA Ames, ISC, DISA DoD NIC, ARL, Netnod, RIPE NCC, ICAN and WIDE). Verisign manages both a.root-servers.net and j.root-servers.net. The number of root server instances changes (generally increases) over time (i.e., 1641 on February 21, 2023). They are deployed at 130 locations across the world. In Belgium, there are 5 root DNS server instances deployed in Brussels. Two instances are operated by ICANN and one by ISC, Netnod, and NASA each. Different operators deploy a different number of instances. More details can be found at https://root-servers.org.

Interactions between DNS servers



University of Antwerp
Faculty of Science

48

There is another important type of DNS server called the **local DNS server**. A local DNS server does not strictly belong to the hierarchy of servers but is nevertheless central to the DNS architecture. Each ISP—such as a residential ISP or an institutional ISP—has a local DNS server (also called a default name server). When a host connects to an ISP, the ISP provides the host with the IP addresses of one or more of its local DNS servers (typically through DHCP). You can easily determine the IP address of your local DNS server by accessing network status windows in Windows or UNIX. A host's local DNS server is typically "close to" the host. For an institutional ISP, the local DNS server may be on the same LAN as the host; for a residential ISP, it is typically separated from the host by no more than a few routers. When a host makes a DNS query, the query is sent to the local DNS server, which acts a proxy, forwarding the query into the DNS server hierarchy.

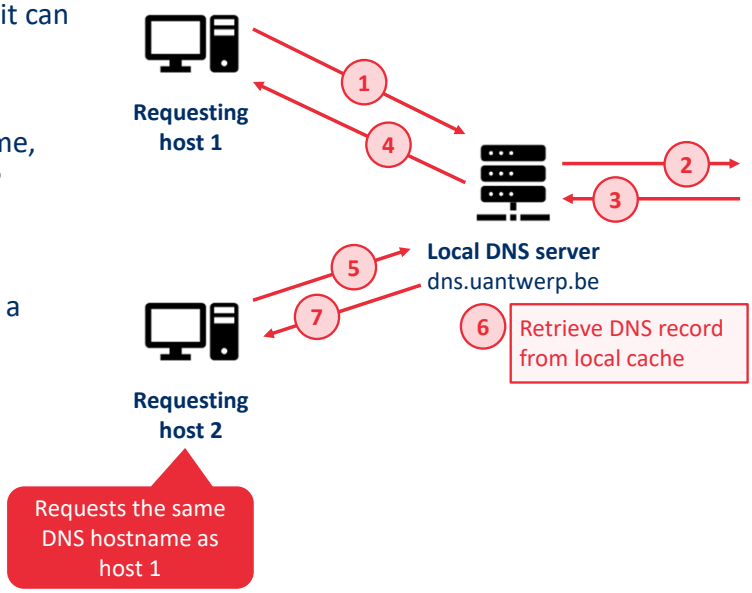
Let's take a look at a simple example. Suppose a host in in the uantwerp.be domain desires the IP address of news.google.com. Also suppose that UAntwerp's local DNS server is called dns.uantwerp.be and that an authoritative DNS server for news.google.com is called dns.google.com. As shown in the host first sends a DNS query message to its local DNS server, dns.uantwerp.be. The query message contains the hostname to be translated, namely, news.google.com. The local DNS server forwards the query message to a root DNS server. The root DNS server takes note of the com suffix and returns to the local DNS server a list of IP addresses for TLD servers responsible for com. The local DNS server then resends the query message to one of these TLD servers. The TLD server takes note of the google.com suffix and responds with the IP address of the authoritative DNS server for Google, namely, dns.google.com. Finally, the local DNS server resends the query message directly to dns.google.com, which responds with the IP address of news.google.com. Note that in this example, in order to obtain the mapping for one hostname, eight DNS messages were sent: four query messages and four reply messages! We'll soon see how DNS caching reduces this query

traffic.

Our example assumed that the TLD server knows the authoritative DNS server for the hostname. In general, this is not always true. Instead, the TLD server may know only of an intermediate DNS server, which in turn knows the authoritative DNS server for the hostname.

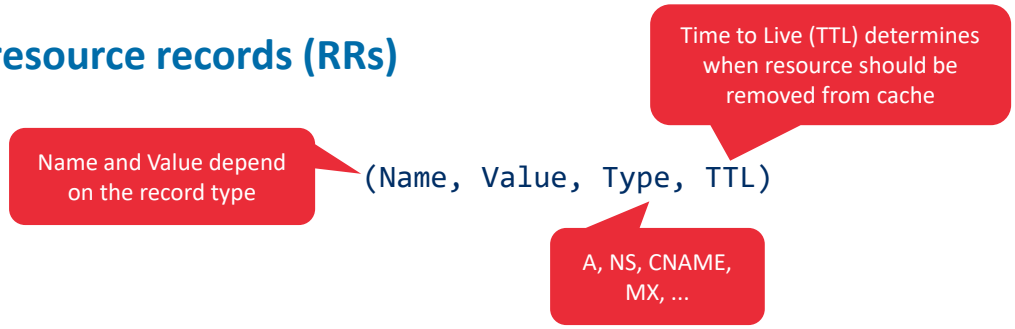
DNS caching

- Whenever a DNS server receives a DNS reply it can cache the mapping in local memory
- If another query arrives for the same hostname, the DNS server can immediately return the IP address
- DNS servers discard cached information after a period of time (usually 2 days)



DNS extensively exploits DNS caching in order to improve the delay performance and to reduce the number of DNS messages ricocheting around the Internet. The idea behind DNS caching is very simple. In a query chain, when a DNS server receives a DNS reply (containing, for example, a mapping from a hostname to an IP address), it can cache the mapping in its local memory. If a hostname/IP address pair is cached in a DNS server and another query arrives to the DNS server for the same hostname, the DNS server can provide the desired IP address, even if it is not authoritative for the hostname. Because hosts and mappings between hostnames and IP addresses are by no means permanent, DNS servers discard cached information after a period of time (often set to two days).

DNS resource records (RRs)



(most important) DNS record types:

- **A**: Provides a standard hostname-to-IP address mapping (Name is hostname and Value is IP address)
- **NS**: Used to route queries along the query chain (Name is domain and Value is authoritative DNS server)
- **CNAME**: Maps alias hostnames (Name) to canonical hostnames (Value)
- **MX**: Maps alias mail server names (Name) to canonical mail server names (Value)

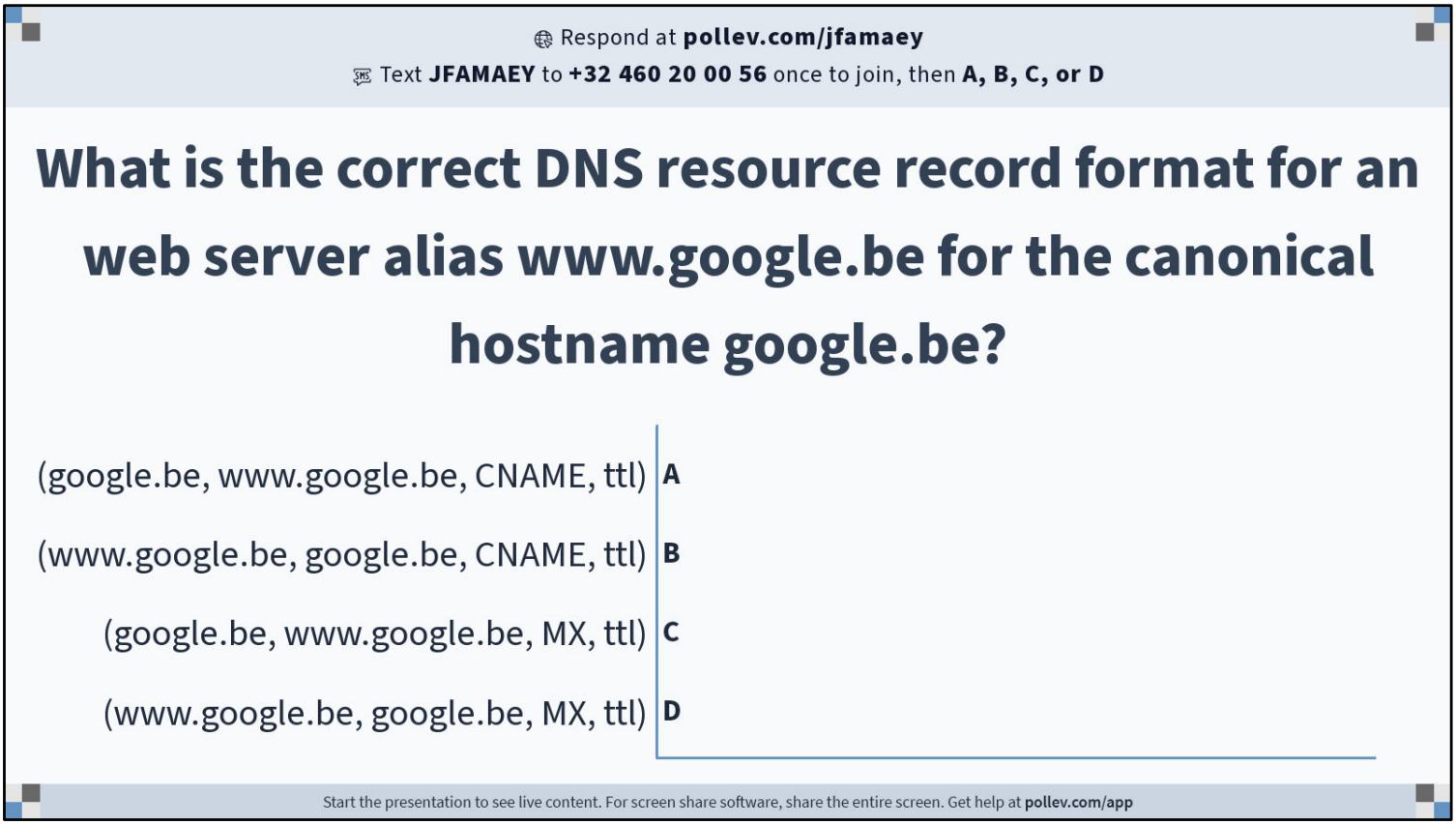
The DNS servers that together implement the DNS distributed database store **resource records (RRs)**, including RRs that provide hostname-to-IP address mappings. Each DNS reply message carries one or more resource records. A resource record is a four-tuple that contains the following fields:

(Name, Value, Type, TTL)

TTL is the time to live of the resource record; it determines when a resource should be removed from a cache. In the example records given below, we ignore the TTL field. The meaning of Name and Value depend on Type:

- If Type=A, then Name is a hostname and Value is the IP address for the hostname. Thus, a Type A record provides the standard hostname-to-IP address mapping. As an example, (relay1.bar.foo.com, 145.37.93.126, A) is a Type A record.
- If Type=NS, then Name is a domain (such as foo.com) and Value is the hostname of an authoritative DNS server that knows how to obtain the IP addresses for hosts in the domain. This record is used to route DNS queries further along in the query chain. As an example, (foo.com, dns.foo.com, NS) is a Type NS record.
- If Type=CNAME, then Value is a canonical hostname for the alias hostname Name. This record can provide querying hosts the canonical name for a hostname. As an example, (foo.com, relay1.bar.foo.com, CNAME) is a CNAME record.
- If Type=MX, then Value is the canonical name of a mail server that has an alias hostname Name. As an example, (foo.com, mail.bar.foo.com, MX) is an MX record. MX records allow the hostnames of mail servers to have simple aliases. Note that by using the MX record, a company can have the same aliased name for its mail server and for one of its other servers (such as its Web server). To obtain the canonical name for the mail server, a DNS client would query for an MX record; to obtain the canonical name for the other server, the DNS client would query for the CNAME record.

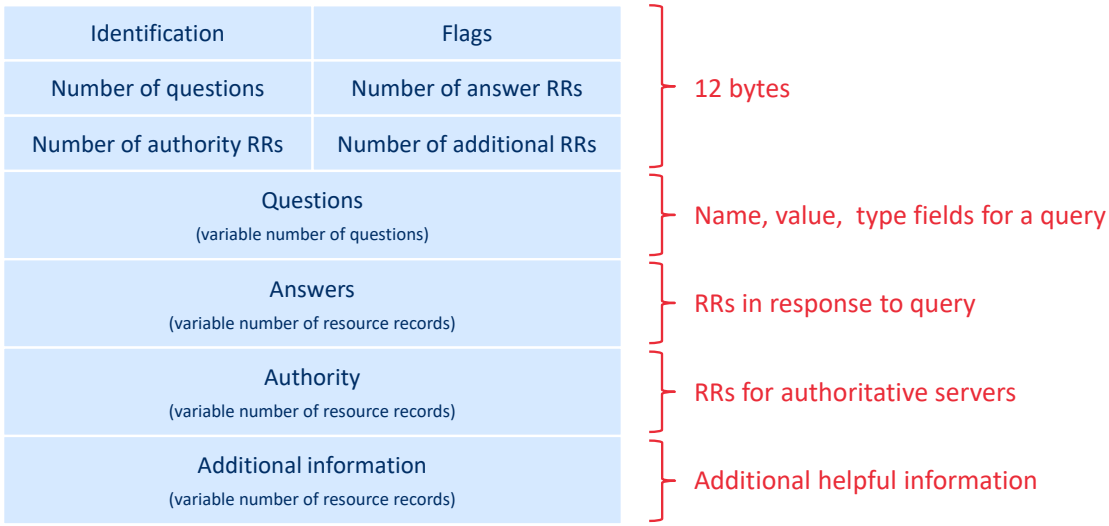
If a DNS server is authoritative for a particular hostname, then the DNS server will contain a Type A record for the hostname. (Even if the DNS server is not authoritative, it may contain a Type A record in its cache.) If a server is not authoritative for a hostname, then the server will contain a Type NS record for the domain that includes the hostname; it will also contain a Type A record that provides the IP address of the DNS server in the Value field of the NS record. As an example, suppose an edu TLD server is not authoritative for the host gaia.cs.umass.edu. Then this server will contain a record for a domain that includes the host gaia.cs.umass.edu, for example, (umass.edu, dns.umass.edu, NS). The edu TLD server would also contain a Type A record, which maps the DNS server dns.umass.edu to an IP address, for example, (dns.umass.edu, 128.119.40.111, A).



Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

What is the correct DNS resource record format for an web server alias www.google.be for the canonical hostname google.be?
https://www.polleverywhere.com/multiple_choice_polls/OvuGhXC8eyFv1m2BYbz0V?state=opened&flow=Default&onscreen=persist

DNS messages (query and reply use the same format)



University of Antwerp
Faculty of Science

53

Earlier in this section, we referred to DNS query and reply messages. These are the only two kinds of DNS messages. Furthermore, both query and reply messages have the same format. The semantics of the various fields in a DNS message are as follows.

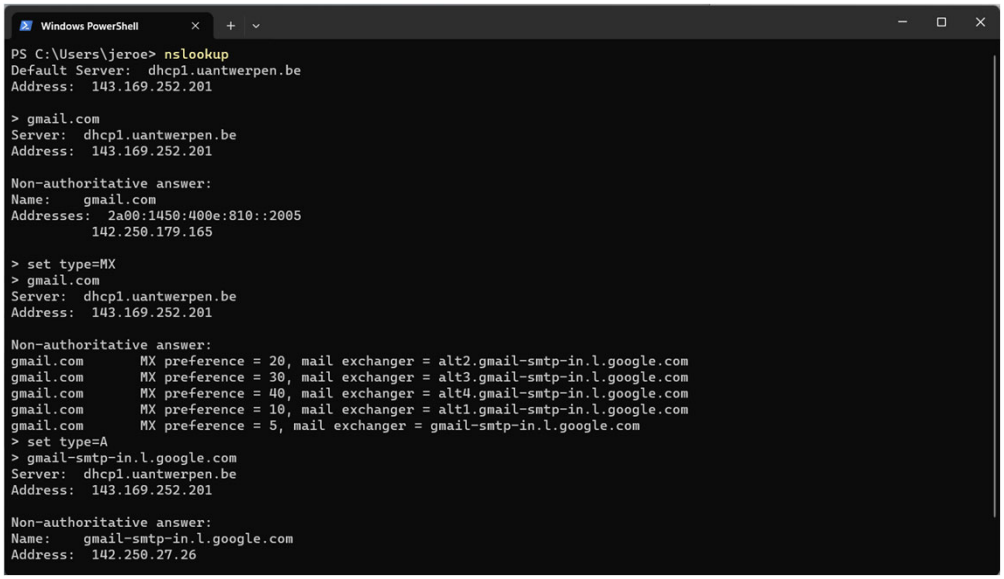
The first 12 bytes is the ***header section***, which has a number of fields. The first field is a 16-bit number that identifies the query. This identifier is copied into the reply message to a query, allowing the client to match received replies with sent queries. There are a number of flags in the flag field. A 1-bit query/reply flag indicates whether the message is a query (0) or a reply (1). A 1-bit authoritative flag is set in a reply message when a DNS server is an authoritative server for a queried name. A 1-bit recursion-desired flag is set when a client (host or DNS server) desires that the DNS server perform recursion when it doesn't have the record. A 1-bit recursion-available field is set in a reply if the DNS server supports recursion. In the header, there are also four number-of fields. These fields indicate the number of occurrences of the four types of data sections that follow the header.

The ***question section*** contains information about the query that is being made. This section includes (1) a name field that contains the name that is being queried, and (2) a type field that indicates the type of question being asked about the name—for example, a host address associated with a name (Type A) or the mail server for a name (Type MX).

In a reply from a DNS server, the ***answer section*** contains the resource records for the name that was originally queried. Recall that in each resource record there is the Type (for example, A, NS, CNAME, and MX), the Value, and the TTL. A reply can return multiple RRs in the answer, since a hostname can have multiple IP addresses (for example, for replicated Web servers, as discussed earlier in this section).

The ***authority section*** contains records of other authoritative servers. The ***additional section*** contains other helpful records. For example, the answer field in a reply to an MX query contains a resource record providing the canonical hostname of a mail server. The additional section contains a Type A record providing the IP address for the canonical hostname of the mail server.

Homework: Performing DNS queries with *nslookup*



How would you like to send a DNS query message directly from the host you're working on to some DNS server? This can easily be done with the **nslookup program**, which is available from most Windows and UNIX platforms. For example, from a Windows host, open the Command Prompt and invoke the nslookup program by simply typing "nslookup." After invoking nslookup, you can send a DNS query to any DNS server (root, TLD, or authoritative). After receiving the reply message from the DNS server, nslookup will display the records included in the reply (in a human-readable format). As an alternative to running nslookup from your own host, you can visit one of many Web sites that allow you to remotely employ nslookup. (Just type "nslookup" into a search engine and you'll be brought to one of these sites.)

How are DNS records inserted into the DNS database?

Steps

1. Register your new domain name at a **registrar**
2. Provide primary and secondary authoritative DNS servers to registrar (or use theirs)
3. Registrar creates a Type NS and Type A record for each of the authoritative DNS servers
4. Enter Type A (for web server) and Type MX (for mail server) record into authoritative DNS servers

Example

dns1.mysite.com
(www.mysite.com, <web server IP>, A)
(mail.mysite.com, <mail server IP>, A)
(mysite.com, mail.mysite.com, MX)

dns2.mysite.com
(www.mysite.com, <web server IP>, A)
(mail.mysite.com, <mail server IP>, A)
(mysite.com, mail.mysite.com, MX)

com TLD DNS server
(mysite.com, dns1.mysite.com, NS)
(mysite.com, dns2.mysite.com, NS)
(dns1.mysite.com, <DNS1 IP>, A)
(dns2.mysite.com, <DNS2 IP>, A)

University of Antwerp | Faculty of Science

55

The discussion above focused on how records are retrieved from the DNS database. You might be wondering how records get into the database in the first place. Let's look at how this is done in the context of a specific example. Suppose you have just created an exciting new startup company called Network Utopia. The first thing you'll surely want to do is register the domain name networkutopia.com at a registrar. A **registrar** is a commercial entity that verifies the uniqueness of the domain name, enters the domain name into the DNS database (as discussed below), and collects a small fee from you for its services. Prior to 1999, a single registrar, Network Solutions, had a monopoly on domain name registration for com, net, and org domains. But now there are many registrars competing for customers, and the Internet Corporation for Assigned Names and Numbers (ICANN) accredits the various registrars. A complete list of accredited registrars is available at http://www.internic.net.

When you register the domain name networkutopia.com with some registrar, you also need to provide the registrar with the names and IP addresses of your primary and secondary authoritative DNS servers. Suppose the names and IP addresses are dns1.networkutopia.com, dns2.networkutopia.com, 212.2.212.1, and 212.212.212.2. For each of these two authoritative DNS servers, the registrar would then make sure that a Type NS and a Type A record are entered into the TLD com servers. Specifically, for the primary authoritative server for networkutopia.com, the registrar would insert the following two resource records into the DNS system:

(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)

You'll also have to make sure that the Type A resource record for your Web server www.networkutopia.com and the Type MX resource record for your mail server

mail.networkutopia.com are entered into your authoritative DNS servers. Until recently, the contents of each DNS server were configured statically, for example, from a configuration file created by a system manager. More recently, an UPDATE option has been added to the DNS protocol to allow data to be dynamically added or deleted from the database via DNS messages. RFC 2136 and RFC 3007 specify DNS dynamic updates.

Video streaming applications and content delivery networks

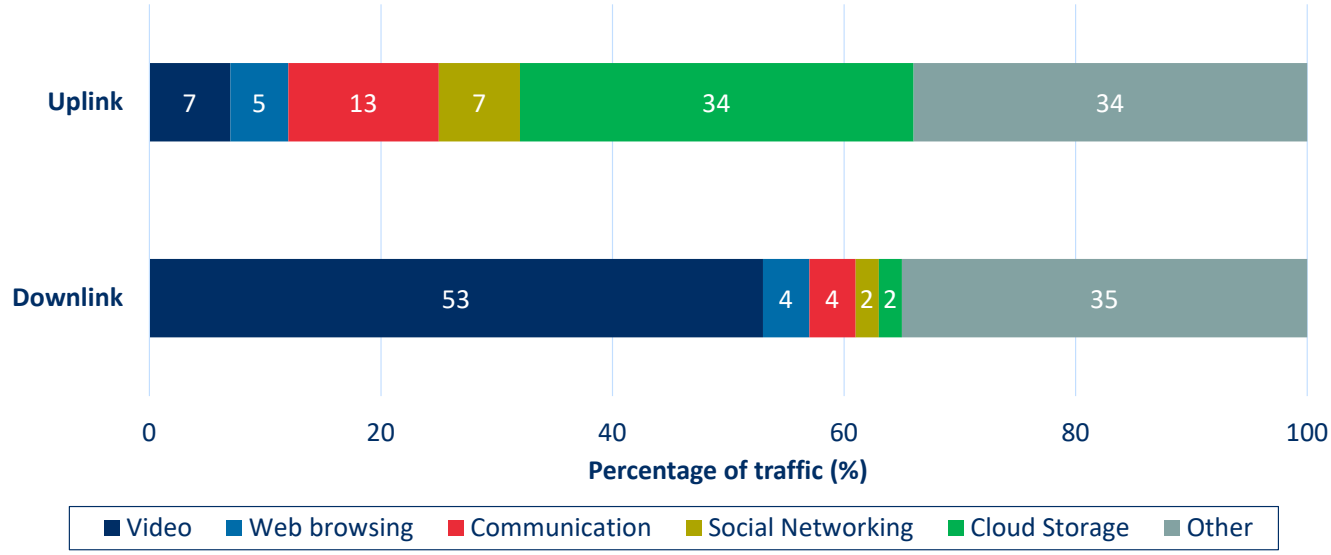
University of Antwerp
Faculty of Science

Computer Networking
A Top-Down Approach

Section 2.6

By many estimates, streaming video—including Netflix, YouTube and Amazon Prime—account for about 80% of Internet traffic in 2020. This part will provide an overview of how popular video streaming services are implemented in today's Internet. We will see they are implemented using application-level protocols and servers that function in some ways like a cache.

Share of traffic per application category for (unnamed) European ISP



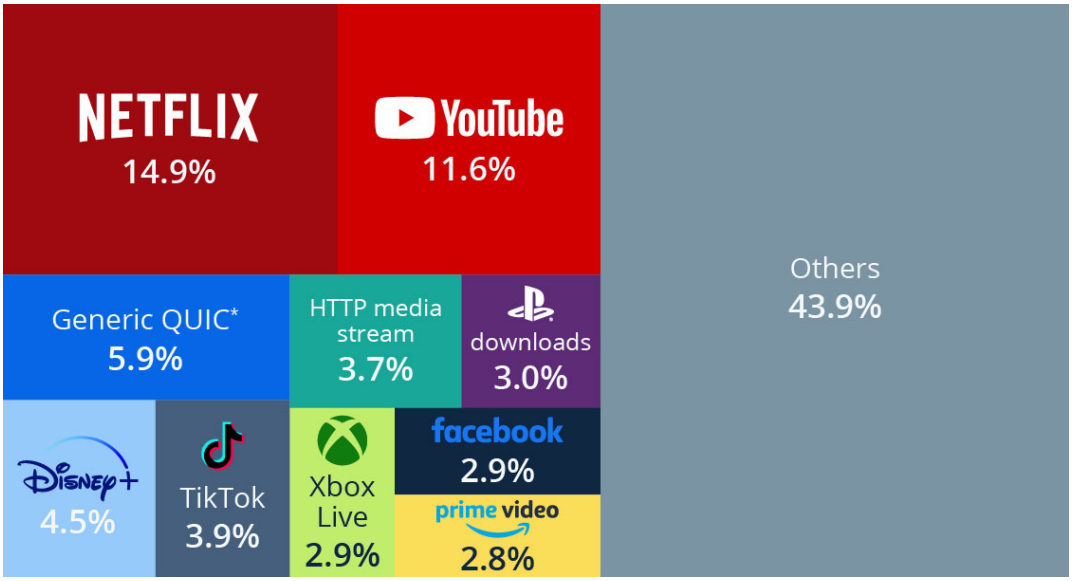
University of Antwerp | Faculty of Science

Source: Ericsson Mobility Report, https://www.ericsson.com/en/reports-and-papers/mobility-report/articles/short-form-dominates-video-traffic

57

In streaming stored video applications, the underlying medium is prerecorded video, such as a movie, a television show, a prerecorded sporting event, or a prerecorded user-generated video (such as those commonly seen on YouTube). These prerecorded videos are placed on servers, and users send requests to the servers to view the videos on demand. Many Internet companies today provide streaming video, including, Netflix, YouTube (Google), Amazon, and TikTok.

Percentage of global Internet traffic per application category in 2022



Source: Statista, "Netflix is Responsible for 15% of Global Internet Traffic," March 2023.

University of Antwerp
Faculty of Science

58

In streaming stored video applications, the underlying medium is prerecorded video, such as a movie, a television show, a prerecorded sporting event, or a prerecorded user-generated video (such as those commonly seen on YouTube). These prerecorded videos are placed on servers, and users send requests to the servers to view the videos on demand. Many Internet companies today provide streaming video, including, Netflix, YouTube (Google), Amazon, and TikTok.

Video characteristics

- A video is a sequence of (encoded/compressed) images
  - Images are displayed at a certain rate, expressed as frames per second (FPS), such as 24, 30, or 60
  - The size of the images (in bytes) determines the bitrate of the video, expressed as (Megabits) bits per second ((M)bps)
- The same video can be encoded in multiple qualities/bitrates (this is the basis of adaptive streaming)

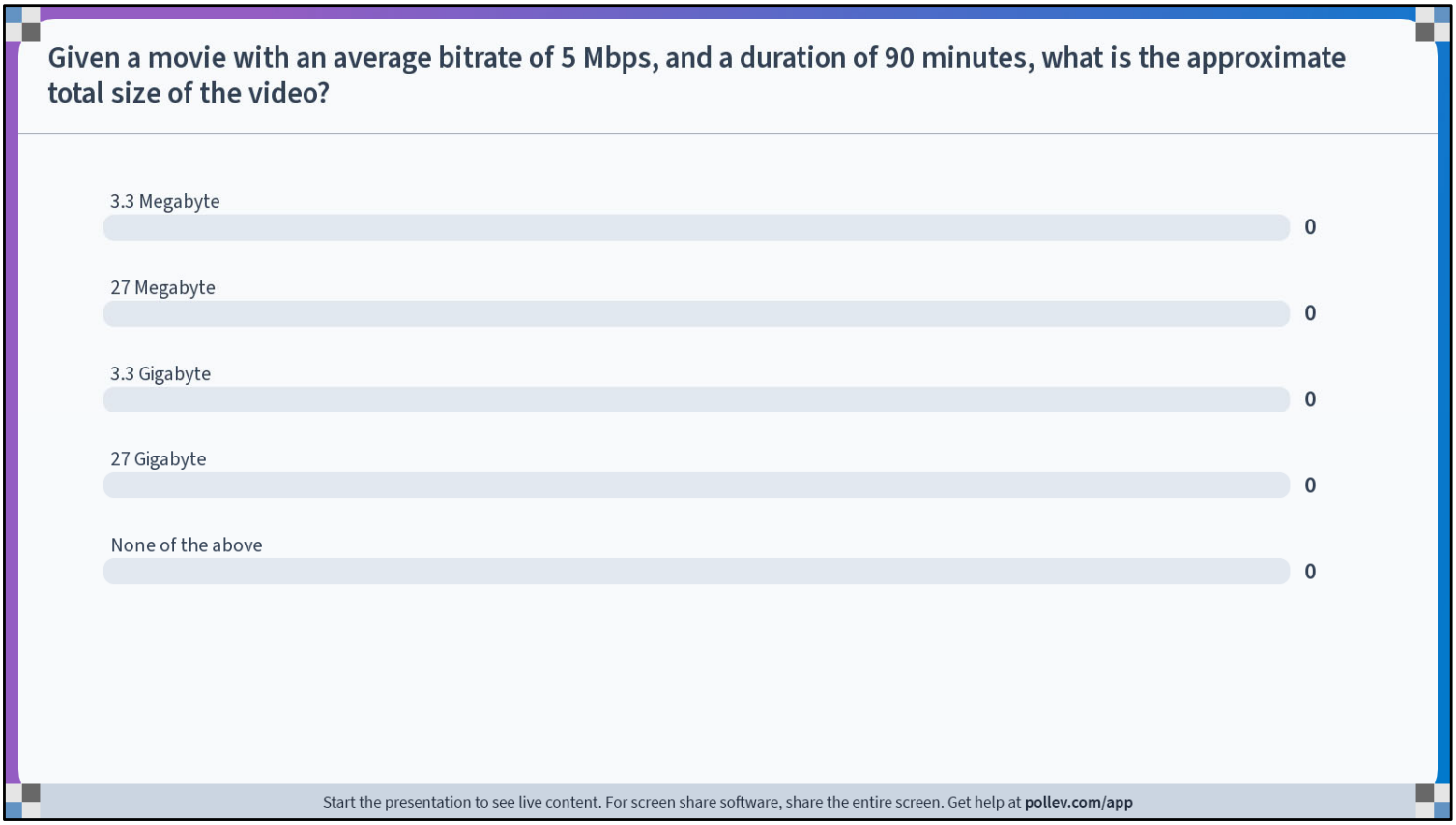
24 FPS, 1080p (1920x1080 pixel), 5 Mbps

24 FPS, 4K (3840x2160 pixel), 20 Mbps

University of Antwerp
Faculty of Science

Before launching into a discussion of video streaming, we should first get a quick feel for the video medium itself. A video is a sequence of images, typically being displayed at a constant rate, for example, at 24 or 30 images per second. An uncompressed, digitally encoded image consists of an array of pixels, with each pixel encoded into a number of bits to represent luminance and color. An important characteristic of video is that it can be compressed, thereby trading off video quality with bit rate. Today's off-the-shelf compression algorithms can compress a video to essentially any bit rate desired. Of course, the higher the bit rate, the better the image quality and the better the overall user viewing experience. From a networking perspective, perhaps the most salient characteristic of video is its high bit rate. Compressed Internet video typically ranges from 100 kbps for low-quality video to over 4 Mbps for streaming high-definition movies; 4K streaming envisions a bitrate of more than 10 Mbps. This can translate to huge amount of traffic and storage, particularly for high-end video. For example, a single 2 Mbps video with a duration of 67 minutes will consume 1 gigabyte of storage and traffic. By far, the most important performance measure for streaming video is average endto- end throughput. In order to provide continuous playout, the network must provide an average throughput to the streaming application that is at least as large as the bit rate of the compressed video.

We can also use compression to create multiple versions of the same video, each at a different quality level. For example, we can use compression to create, say, three versions of the same video, at rates of 300 kbps, 1 Mbps, and 3 Mbps. Users can then decide which version they want to watch as a function of their current available bandwidth. Users with high-speed Internet connections might choose the 3 Mbps version; users watching the video over 3G with a smartphone might choose the 300 kbps version.

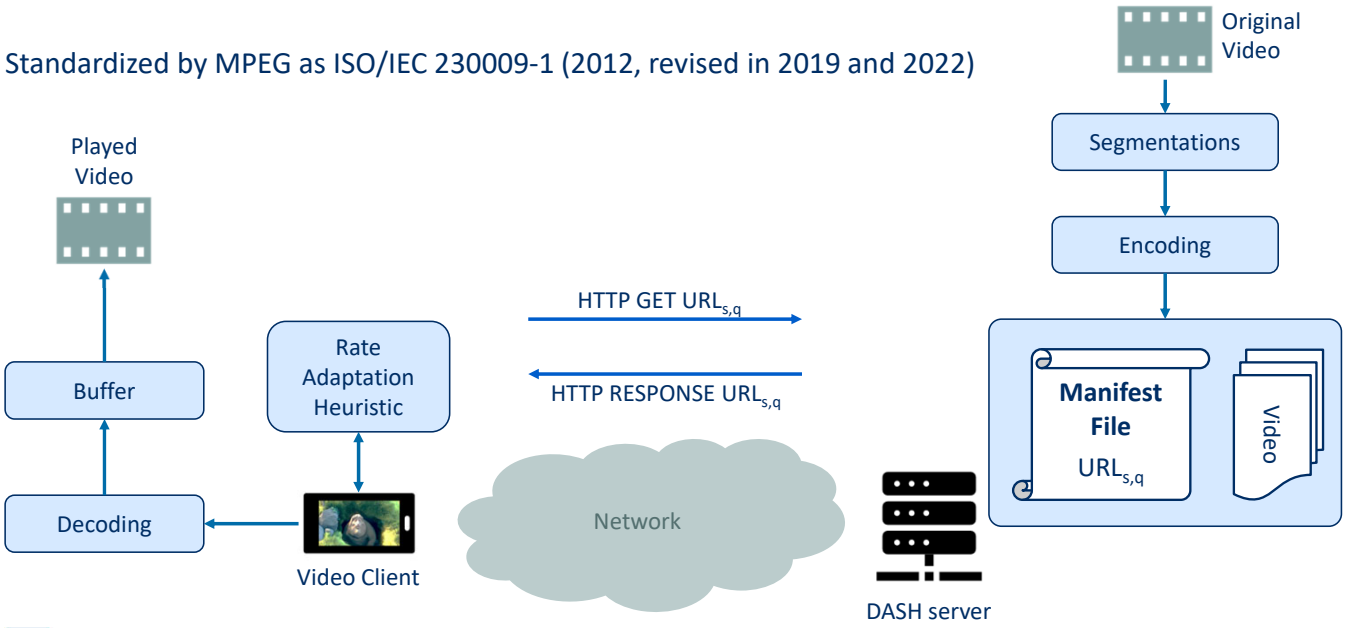


Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

Given a movie with an average bitrate of 5 Mbps, and a duration of 90 minutes, what is the approximate total size of the video?
https://www.polleverywhere.com/multiple_choice_polls/er0K8CcLLpIBbbQEUAtNI?state=opened&flow=Default&onscreen=persist

Dynamic Adaptive Streaming over HTTP (DASH)

Standardized by MPEG as ISO/IEC 230009-1 (2012, revised in 2019 and 2022)



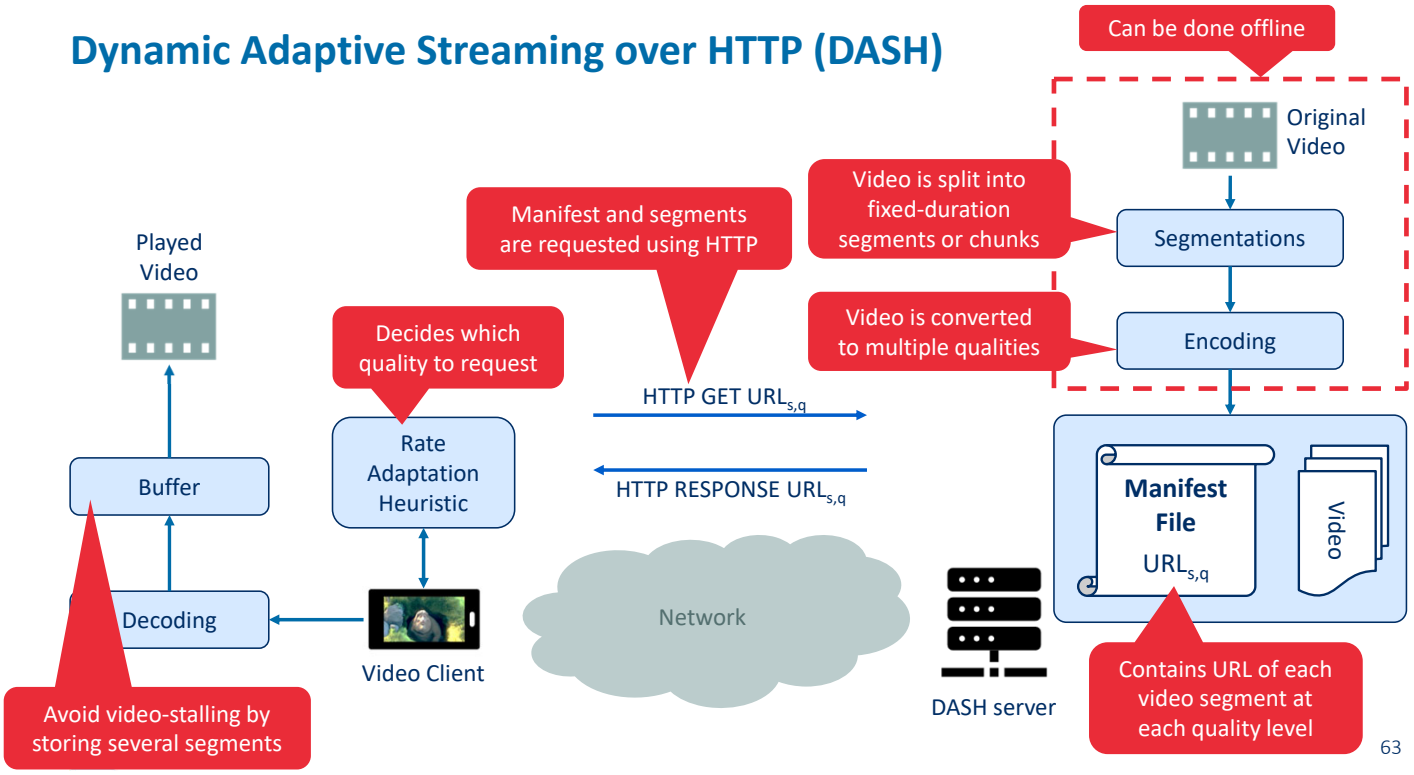
University of Antwerp | Faculty of Science

62

In HTTP streaming, the video is simply stored at an HTTP server as an ordinary file with a specific URL. When a user wants to see the video, the client establishes a TCP connection with the server and issues an HTTP GET request for that URL. The server then sends the video file, within an HTTP response message, as quickly as the underlying network protocols and traffic conditions will allow. On the client side, the bytes are collected in a client application buffer. Once the number of bytes in this buffer exceeds a predetermined threshold, the client application begins playback— specifically, the streaming video application periodically grabs video frames from the client application buffer, decompresses the frames, and displays them on the user's screen. Thus, the video streaming application is displaying video as it is receiving and buffering frames corresponding to latter parts of the video.

Although HTTP streaming, as described in the previous paragraph, has been extensively deployed in practice (for example, by YouTube since its inception), it has a major shortcoming: All clients receive the same encoding of the video, despite the large variations in the amount of bandwidth available to a client, both across different clients and also over time for the same client. This has led to the development of a new type of HTTP-based streaming, often referred to as Dynamic Adaptive Streaming over HTTP (DASH). In DASH, the video is encoded into several different versions, with each version having a different bit rate and, correspondingly, a different quality level. The client dynamically requests chunks of video segments of a few seconds in length. When the amount of available bandwidth is high, the client naturally selects chunks from a high-rate version; and when the available bandwidth is low, it naturally selects from a low-rate version. The client selects different chunks one at a time with HTTP GET request messages.

Dynamic Adaptive Streaming over HTTP (DASH)

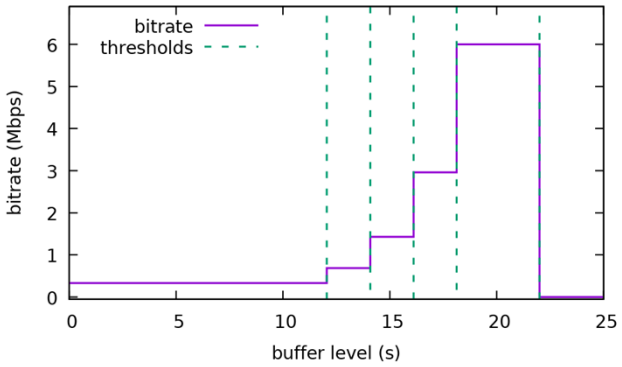


63

Rate Adaptation Heuristic

- Rate adaptation heuristics are “vendor specific” (i.e., they are not standardized)
- Goal: Find a balance between two important Quality of Experience metrics:
  - The **video quality** should be maximized (and (optionally) fluctuations in quality minimized)
  - The **rebuffering time** (i.e., waiting time due to an empty playout buffer) should be minimized
- Information that can be used by the client:
  - Buffer level (i.e., playout duration of buffered segments)
  - Estimation of historical bandwidth (i.e., by dividing segment size by segment download time)
  - Previously requested quality levels

Example of selected video bitrate based on buffer level thresholds ((Buffer Occupancy based Lyapunov Algorithm):



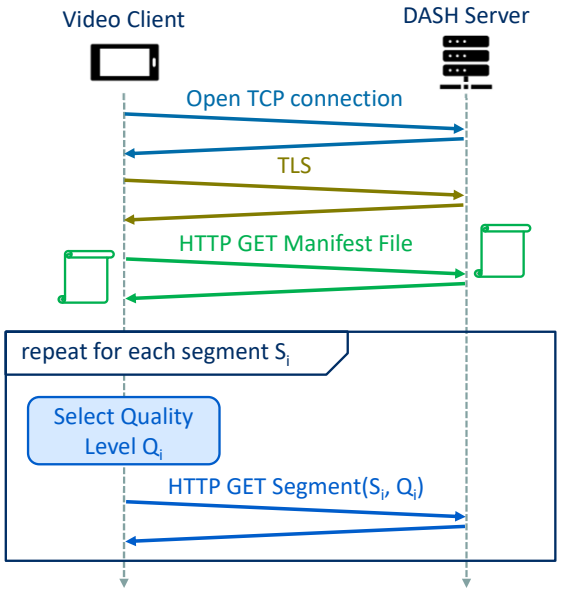
University of Antwerp | Faculty of Science

Source: *K. Spiteri, R. Urgaonkar and R. K. Sitaraman, "BOLA: Near-Optimal Bitrate Adaptation for Online Videos," in IEEE/ACM Transactions on Networking, vol. 28, no. 4, pp. 1698-1711, Aug. 2020, doi: 10.1109/TNET.2020.2996964.*

64

DASH allows clients with different Internet access rates to stream in video at different encoding rates. Clients with low-speed 3G connections can receive a low bit-rate (and low-quality) version, and clients with fiber connections can receive a high-quality version. DASH also allows a client to adapt to the available bandwidth if the available end-to-end bandwidth changes during the session. This feature is particularly important for mobile users, who typically see their bandwidth availability fluctuate as they move with respect to the base stations.

Typical HTTP message exchange for DASH



Example DASH manifest file (MD):

```
<?xml version="1.0" encoding="UTF-8"?>
<MPD xmlns="urn:mpeg:dash:schema:mpd:2011" minBufferTime="PT1.500000S" type="static"
mediaPresentationDuration="PT0H9M56.46S" profiles="urn:mpeg:dash:profile:isoff-
live:2011">
 <ProgramInformation moreInformationURL="http://gpac.sourceforge.net">
  <Title>dashed/BigBuckBunny_10s_simple_2014_05_09.mpd generated by GPAC</Title>
 </ProgramInformation>
 <Period duration="PT0H9M56.46S">
  <AdaptationSet segmentAlignment="true" group="1" maxWidth="480" maxHeight="360"
     maxFrameRate="24" par="4:3">
   <SegmentTemplate timescale="96" media="$Bandwidth$bps/BigBuckBunny_10s$Number$.m4s"
      startNumber="1" duration="960"
      initialization="bunny_$Bandwidth$bps/BigBuckBunny_10s_init.mp4" />
   <Representation id="320x240 45.0kbps" mimeType="video/mp4" codecs="avc1.42c00d"
      width="320" height="240" frameRate="24" startWithSAP="1" bandwidth="45373" />
   <Representation id="1280x720 987.0kbps" mimeType="video/mp4" codecs="avc1.42c01f"
      width="1280" height="720" frameRate="24" startWithSAP="1" bandwidth="987061" />
   <Representation id="1920x1080 3.8Mbps" mimeType="video/mp4" codecs="avc1.42c032"
      width="1920" height="1080" frameRate="24" startWithSAP="1" bandwidth="3792491" />
  </AdaptationSet>
 </Period>
</MPD>
```

University of Antwerp | Faculty of Science

65

With DASH, each video version is stored in the HTTP server, each with a different URL. The HTTP server also has a **manifest file**, which provides a URL for each version along with its bit rate. The client first requests the manifest file and learns about the various versions. The client then selects one chunk at a time by specifying a URL and a byte range in an HTTP GET request message for each chunk. While downloading chunks, the client also measures the received bandwidth and runs a rate determination algorithm to select the chunk to request next. Naturally, if the client has a lot of video buffered and if the measured receive bandwidth is high, it will choose a chunk from a high-bitrate version. And naturally if the client has little video buffered and the measured received bandwidth is low, it will choose a chunk from a low-bitrate version. DASH therefore allows the client to freely switch among different quality levels.

Scalable video streaming using Content Delivery Networks (CDNs)

Naïve approach: A single centralized datacenter

Disadvantages:

- High delay (packets cross many links and ISPs)
- Popular videos are sent many times over the same links
- Single point-of-failure

Scalable approach: Distributed CDN

- Types of CDNs
  - **Private CDN**: Owned by a content provider (e.g., Netflix, Google)
  - **Third-party CDN**: Distributes on behalf of a provider (e.g., Akamai)
- Server placement philosophies
  - **Enter Deep:** Deploy servers in ISP access networks (near end users)
  - **Bring Home**: Build larger clusters at Internet Exchange Points (IXPs)

University of Antwerp | Faculty of Science

66

Today, many Internet video companies are distributing on-demand multi-Mbps streams to millions of users on a daily basis. YouTube, for example, with a library of hundreds of millions of videos, distributes hundreds of millions of video streams to users around the world every day. Streaming all this traffic to locations all over the world while providing continuous playout and high interactivity is clearly a challenging task.

For an Internet video company, perhaps the most straightforward approach to providing streaming video service is to build a single massive data center, store all of its videos in the data center, and stream the videos directly from the data center to clients worldwide. But there are three major problems with this approach. First, if the client is far from the data center, server-to-client packets will cross many communication links and likely pass through many ISPs, with some of the ISPs possibly located on different continents. If one of these links provides a throughput that is less than the video consumption rate, the end-to-end throughput will also be below the consumption rate, resulting in annoying freezing delays for the user. (Recall from Chapter 1 that the end-to-end throughput of a stream is governed by the throughput at the bottleneck link.) The likelihood of this happening increases as the number of links in the end-to-end path increases. A second drawback is that a popular video will likely be sent many times over the same communication links. Not only does this waste network bandwidth, but the Internet video company itself will be paying its provider ISP (connected to the data center) for sending the same bytes into the Internet over and over again. A third problem with this solution is that a single data center represents a single point of failure—if the data center or its links to the Internet goes down, it would not be able to distribute any video streams.

In order to meet the challenge of distributing massive amounts of video data to users distributed around the world, almost all major video-streaming companies make use of **Content Distribution Networks (CDNs)**. A CDN manages servers in multiple geographically distributed

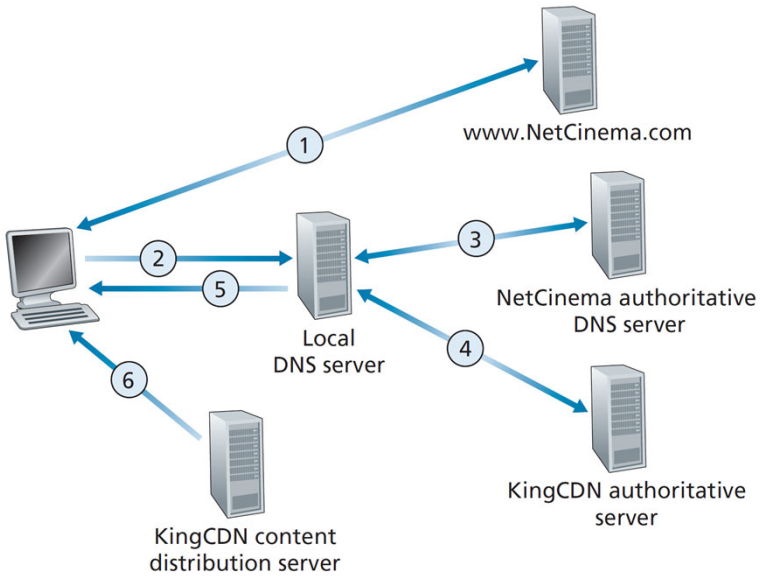
locations, stores copies of the videos (and other types of Web content, including documents, images, and audio) in its servers, and attempts to direct each user request to a CDN location that will provide the best user experience. The CDN may be a **private CDN**, that is, owned by the content provider itself; for example, Google's CDN distributes YouTube videos and other types of content. The CDN may alternatively be a **third-party CDN** that distributes content on behalf of multiple content providers; Akamai, Limelight and Level-3 all operate third-party CDNs.

CDNs typically adopt one of two different server placement philosophies:

- **Enter Deep.** One philosophy, pioneered by Akamai, is to enter deep into the access networks of Internet Service Providers, by deploying server clusters in access ISPs all over the world. (Access networks are described in Section 1.3.) Akamai takes this approach with clusters in thousands of locations. The goal is to get close to end users, thereby improving user-perceived delay and throughput by decreasing the number of links and routers between the end user and the CDN server from which it receives content. Because of this highly distributed design, the task of maintaining and managing the clusters becomes challenging.
- **Bring Home.** A second design philosophy, taken by Limelight and many other CDN companies, is to bring the ISPs home by building large clusters at a smaller number (for example, tens) of sites. Instead of getting inside the access ISPs, these CDNs typically place their clusters in Internet Exchange Points (IXPs). Compared with the enter-deep design philosophy, the bring-home design typically results in lower maintenance and management overhead, possibly at the expense of higher delay and lower throughput to end users.

Once its clusters are in place, the CDN replicates content across its clusters. The CDN may not want to place a copy of every video in each cluster, since some videos are rarely viewed or are only popular in some countries. In fact, many CDNs do not push videos to their clusters but instead use a simple pull strategy: If a client requests a video from a cluster that is not storing the video, then the cluster retrieves the video (from a central repository or from another cluster) and stores a copy locally while streaming the video to the client at the same time.

CDN operation based on DNS interception and redirection



1. User visits NetCinema webpage
2. DNS request for selected video hostname is sent to local DNS server
3. NetCinema DNS server detects it's a video URL and forwards to KingCDN
4. KingCDN private DNS infrastructure forwards user to its own content distribution server
5. Local DNS server forwards IP address of KingCDN content server to user host
6. User downloads content from KingCDN content server via HTTP

University of Antwerp | Faculty of Science

67

Having identified the two major approaches toward deploying a CDN, let's now dive down into the nuts and bolts of how a CDN operates. When a browser in a user's host is instructed to retrieve a specific video (identified by a URL), the CDN must intercept the request so that it can (1) determine a suitable CDN server cluster for that client at that time, and (2) redirect the client's request to a server in that cluster. We'll shortly discuss how a CDN can determine a suitable cluster. But first let's examine the mechanics behind intercepting and redirecting a request.

Most CDNs take advantage of DNS to intercept and redirect requests. Let's consider a simple example to illustrate how the DNS is typically involved. Suppose a content provider, NetCinema, employs the third-party CDN company, KingCDN, to distribute its videos to its customers. On the NetCinema Web pages, each of its videos is assigned a URL that includes the string "video" and a unique identifier for the video itself; for example, Transformers 7 might be assigned http://video.netcinema.com/6Y7B23V. Six steps then occur:

1. The user visits the Web page at NetCinema.
2. When the user clicks on the link http://video.netcinema.com/6Y7B23V, the user's host sends a DNS query for video.netcinema.com.
3. The user's Local DNS Server (LDNS) relays the DNS query to an authoritative DNS server for NetCinema, which observes the string "video" in the hostname video.netcinema.com. To "hand over" the DNS query to KingCDN, instead of returning an IP address, the NetCinema authoritative DNS server returns to the LDNS a hostname in the KingCDN's domain, for example, a1105.kingcdn.com.
4. From this point on, the DNS query enters into KingCDN's private DNS infrastructure. The user's LDNS then sends a second query, now for a1105.kingcdn.com, and KingCDN's DNS system eventually returns the IP addresses of a KingCDN content server to the LDNS. It is thus here, within the KingCDN's DNS system, that the CDN server from which the client will

receive its content is specified.

5. The LDNS forwards the IP address of the content-serving CDN node to the user's host.
6. Once the client receives the IP address for a KingCDN content server, it establishes a direct TCP connection with the server at that IP address and issues an HTTP GET request for the video. If DASH is used, the server will first send to the client a manifest file with a list of URLs, one for each version of the video, and the client will dynamically select chunks from the different versions.

Cluster selection strategy

- Based on the IP address of the client's local DNS server
- Strategy is usually *proprietary*
- Common strategies:

Geographical closeness	Real-time measurements

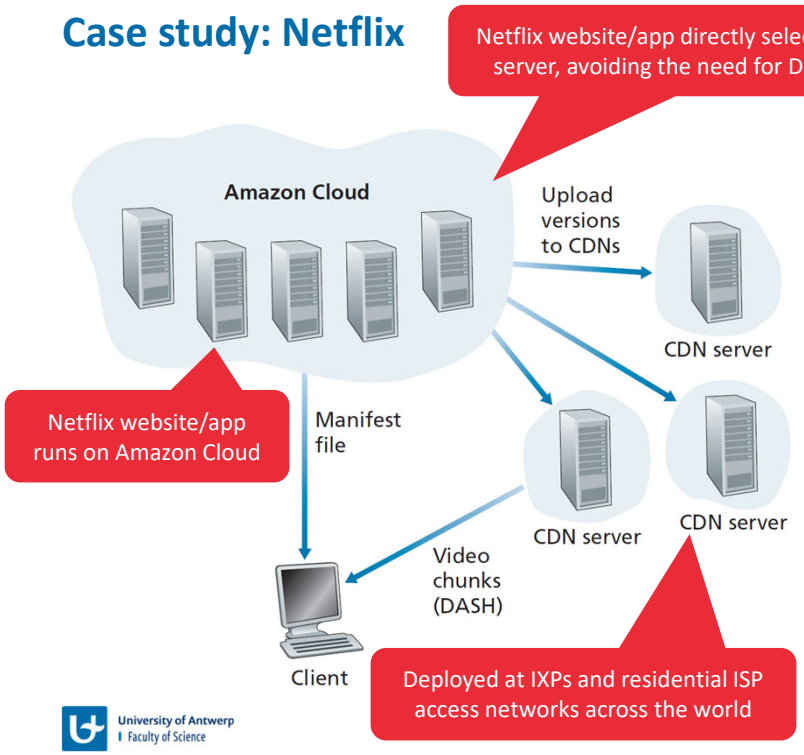
▪ Select content server geographically nearest to the local DNS server
▪ Geo-location databases map IP addresses to geographical locations
▪ Disadvantages
  ▪ Geographic closeness may not reflect network closeness
  ▪ Some users use remotely located local DNS servers
  ▪ Ignores variations in delay and bandwidth over time
	▪ CDN clusters periodically measure delay and loss to local DNS servers using probes (e.g., ping messages)
	▪ Disadvantages
	  ▪ Local DNS servers may not be configured to respond to probes
	  ▪ Significant overhead to frequently ping all possible local DNS servers

At the core of any CDN deployment is a cluster selection strategy, that is, a mechanism for dynamically directing clients to a server cluster or a data center within the CDN. As we just saw, the CDN learns the IP address of the client's LDNS server via the client's DNS lookup. After learning this IP address, the CDN needs to select an appropriate cluster based on this IP address. CDNs generally employ proprietary cluster selection strategies. We now briefly survey a few approaches, each of which has its own advantages and disadvantages.

One simple strategy is to assign the client to the cluster that is **geographically closest**. Using commercial geo-location databases, each LDNS IP address is mapped to a geographic location. When a DNS request is received from a particular LDNS, the CDN chooses the geographically closest cluster, that is, the cluster that is the fewest kilometers from the LDNS "as the bird flies." Such a solution can work reasonably well for a large fraction of the clients. However, for some clients, the solution may perform poorly, since the geographically closest cluster may not be the closest cluster in terms of the length or number of hops of the network path. Furthermore, a problem inherent with all DNS-based approaches is that some end-users are configured to use remotely located LDNSs, in which case the LDNS location may be far from the client's location. Moreover, this simple strategy ignores the variation in delay and available bandwidth over time of Internet paths, always assigning the same cluster to a particular client.

In order to determine the best cluster for a client based on the *current* traffic conditions, CDNs can instead perform periodic **real-time measurements** of delay and loss performance between their clusters and clients. For instance, a CDN can have each of its clusters periodically send probes (for example, ping messages or DNS queries) to all of the LDNSs around the world. One drawback of this approach is that many LDNSs are configured to not respond to such probes.

Case study: Netflix



Amazon Cloud functions

- Netflix website (user management, billing, movie catalogue)
- Content ingestion (studio master versions)
- Content processing (transform master into diverse bit rates for different devices)
- Uploading content versions to private CDN

Netflix private CDN

- Deployed at > 200 IXP locations (contain full Netflix catalogue)
- Residential ISPs can deploy a (free) Netflix server rack (contain popular content)
- Content is pushed during off-peak hours

University of Antwerp | Faculty of Science

69

As of 2020, Netflix is the leading service provider for online movies and TV series in North America. As we discuss below, Netflix video distribution has two major components: the Amazon cloud and its own private CDN infrastructure. Netflix has a Web site that handles numerous functions, including user registration and login, billing, movie catalogue for browsing and searching, and a movie recommendation system. This web site (and its associated backend databases) run entirely on Amazon servers in the Amazon cloud. Additionally, the Amazon cloud handles the following critical functions:

- **Content ingestion.** Before Netflix can distribute a movie to its customers, it must first ingest and process the movie. Netflix receives studio master versions of movies and uploads them to hosts in the Amazon cloud.
- **Content processing.** The machines in the Amazon cloud create many different formats for each movie, suitable for a diverse array of client video players running on desktop computers, smartphones, and game consoles connected to televisions. A different version is created for each of these formats and at multiple bit rates, allowing for adaptive streaming over HTTP using DASH.
- **Uploading versions to its CDN**. Once all of the versions of a movie have been created, the hosts in the Amazon cloud upload the versions to its CDN.

When Netflix first rolled out its video streaming service in 2007, it employed three third-party CDN companies to distribute its video content. Netflix has since created its own private CDN, from which it now streams all of its videos. To create its own CDN, Netflix has installed server racks both in IXPs and within residential ISPs themselves. Netflix currently has server racks in over 200 IXP locations. There are also hundreds of ISP locations housing Netflix racks, where Netflix provides to potential ISP partners instructions about installing a (free) Netflix rack for their networks. Each server in the rack has several 10 Gbps Ethernet ports and over 100 terabytes of storage. The number of servers in a rack varies: IXP installations often have tens of

servers and contain the entire Netflix streaming video library, including multiple versions of the videos to support DASH. Netflix does not use pull-caching to populate its CDN servers in the IXPs and ISPs. Instead, Netflix distributes by pushing the videos to its CDN servers during off-peak hours. For those locations that cannot hold the entire library, Netflix pushes only the most popular videos, which are determined on a day-to-day basis.

Having described the components of the Netflix architecture, let's take a closer look at the interaction between the client and the various servers that are involved in movie delivery. As indicated earlier, the Web pages for browsing the Netflix video library are served from servers in the Amazon cloud. When a user selects a movie to play, the Netflix software, running in the Amazon cloud, first determines which of its CDN servers have copies of the movie. Among the servers that have the movie, the software then determines the "best" server for that client request. If the client is using a residential ISP that has a Netflix CDN server rack installed in that ISP, and this rack has a copy of the requested movie, then a server in this rack is typically selected. If not, a server at a nearby IXP is typically selected.

Netflix has been able to simplify and tailor its CDN design. In particular, Netflix does not need to employ DNS redirect to connect a particular client to a CDN server; instead, the Netflix software (running in the Amazon cloud) directly tells the client to use a particular CDN server.

Summary

University of Antwerp
Faculty of Science

Summary

- The **application layer** consists of network applications and application-layer protocols
  - The run only on end hosts (and not on routers or switches)
  - Sockets provide an API to transmit or receive data via the transport layer (e.g., UDP or TCP)
- The **HyperText Transfer Protocol (HTTP)** enables communication between web clients and servers
  - It makes use of TCP to ensure reliable end-to-end communication
  - HTTP requests are used to request web pages and objects, responses transfer objects back to the client
  - Persistent connections and pipelining reduce latency
  - HTTP/2 offers additional features (multiplexing, prioritization, push and compression) to further reduce perceived latency
- The **Simple Mail Transfer Protocol (SMTP)** enables communication between mail servers
  - It is a push-based protocol for transmitting email messages towards the recipients mail server
  - Other protocols are needed to retrieve email from the mail server's mailbox (e.g., HTTP or IMAP)
- The **Domain Name System (DNS)** maps human-readable hostnames to router-readable IP addresses
  - It is based on a hierarchy of DNS servers
  - It also offers aliasing and load distribution functionality
- **Video streaming applications** use HTTP in combination with Content Delivery Networks for scalable content delivery
- The contents of this part is covered in the **book** in Sections 2.1, 2.2, 2.3, 2.4, and 2.6

University of Antwerp
Faculty of Science

End of Part 2